Cyber Physical Robotics

Dr. Paulo Garcia

2026

Cyber Physical Robotics

Today:

- Real-time systems
- Scheduling theory

Cyber Physical Robotics

The goal

- To be able to have a robot swarm operating concurrently
  - Robots operate asynchronously
  - Each robot may be performing a different task
  - **When** things happen matters
- Let's start with a simpler problem: a single system (not concurrent), scheduling different tasks
  - A (single-core) processor running several software processes
  - A (single)robot that must perform different tasks

Cyber Physical Robotics

- What does "real-time" mean?

Cyber Physical Robotics

A system is a real-time system if:

- There is a deadline for executing at least one of its tasks
  - At some point in time, if task is not complete, the system has failed

Cyber Physical Robotics

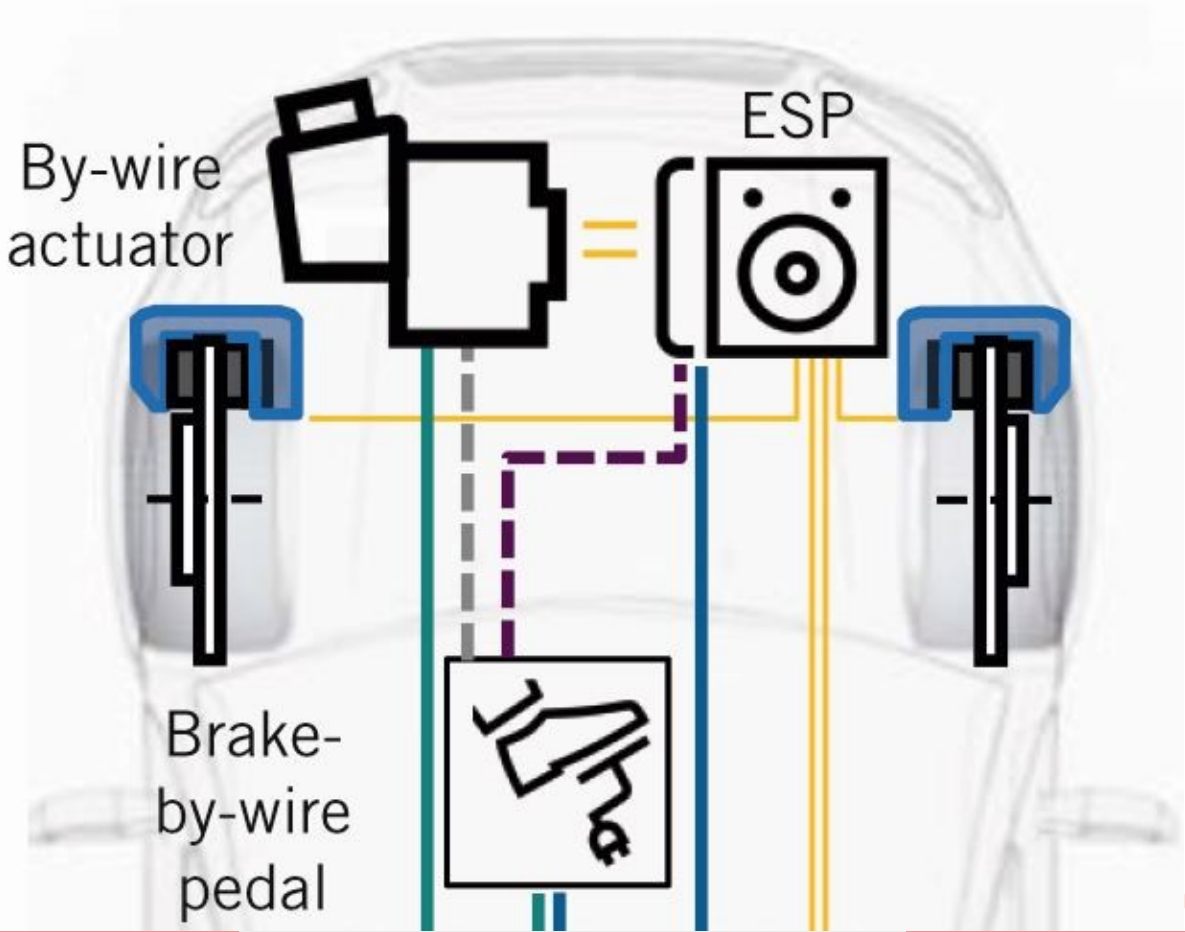
Two types:

- **Hard real-time**: failing to meet a deadline means the system has **failed**
- **Soft real-time**: failing to meet a deadline means the system has **performed poorly**

Cyber Physical Robotics

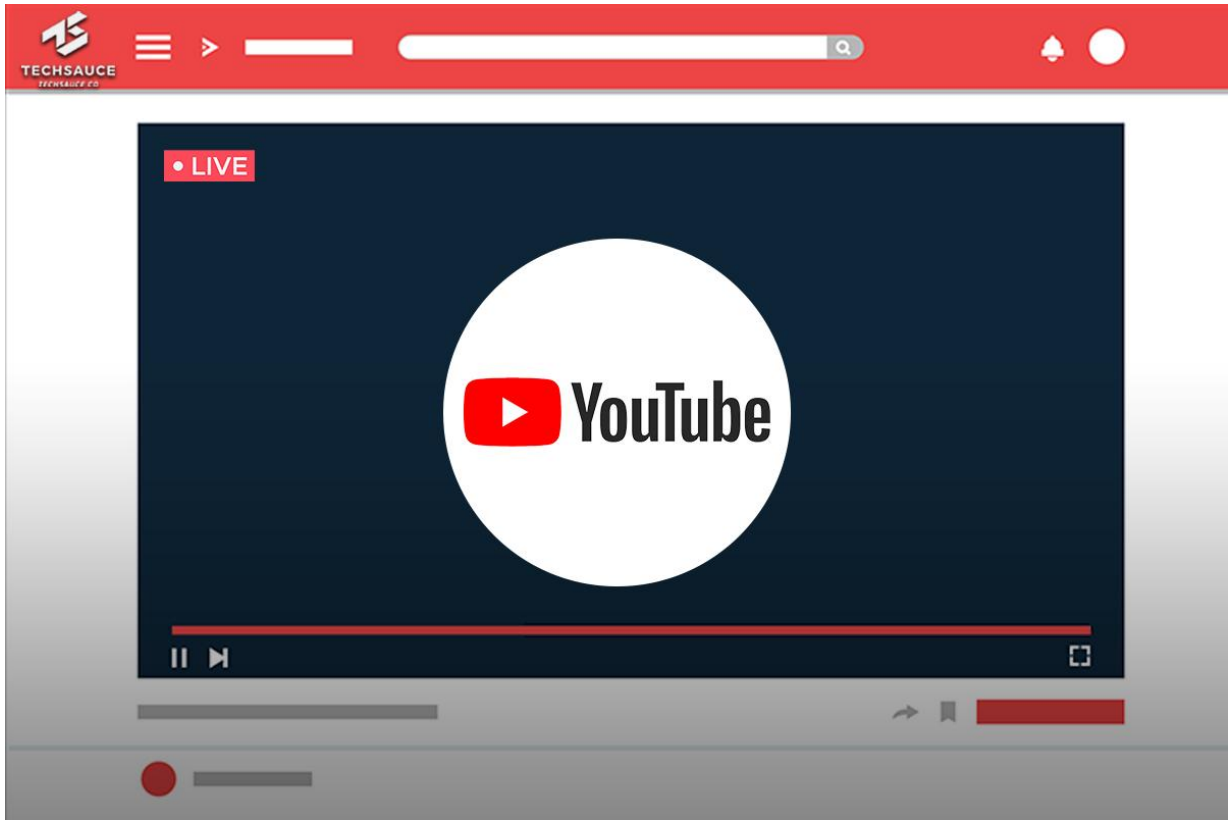
Example of hard real-time

- Car brake system: maximum delay betweeb pressing the pedal and actuator working is 100ms



Example of soft real-time

- Video streaming



Cyber Physical Robotics

(depending on video quality) 30 frames per second (FPS)

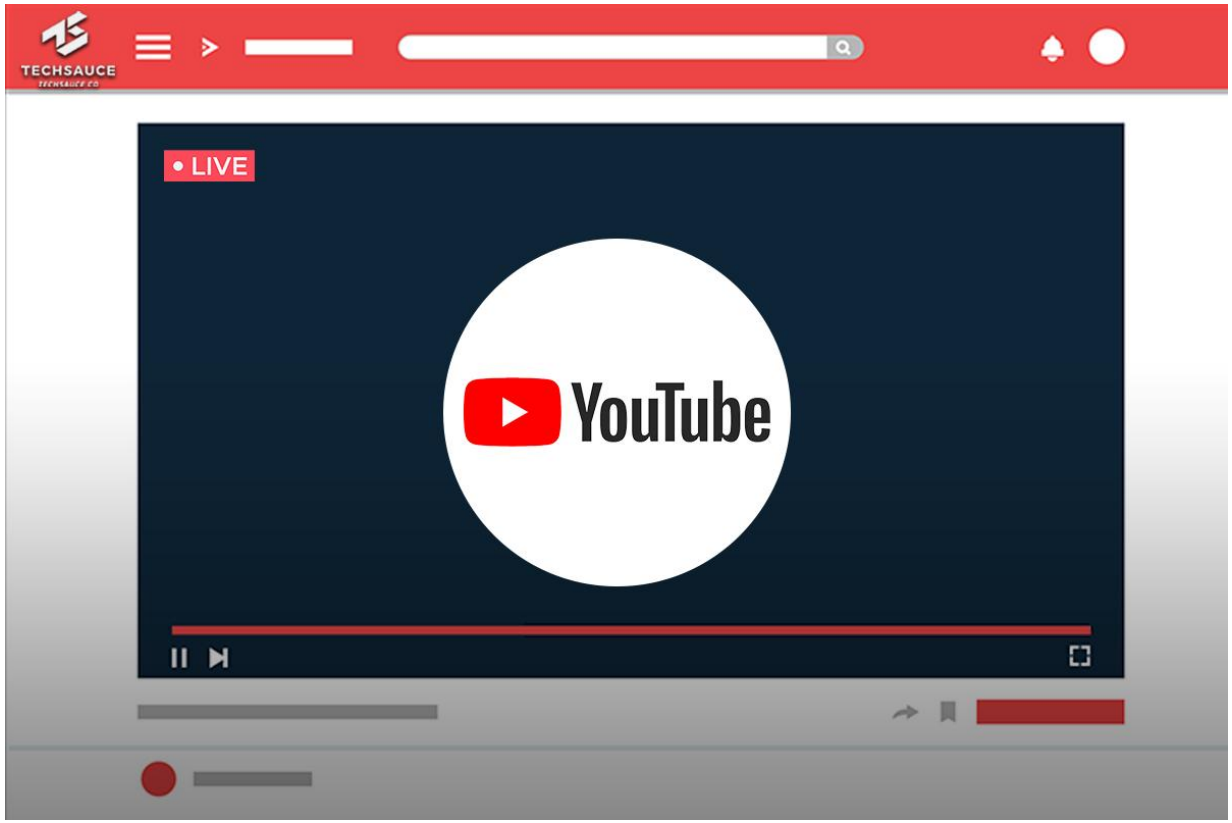
- each frame must be downloaded and rendered...
- ... in less than 1/30 seconds

Otherwise, streaming quality suffers (missing frames, slowdown...)

Cyber Physical Robotics

Even if you could do it faster

- You shouldn't! That's a sped up video
- Doing it too fast ruins the experience



Real-time systems

More formal definitions:

- **Periodic tasks**:
  - Must be executed repeatedly
  - Have an arrival time (beginning of period)
  - Have a deadline (end of period)

Real-time systems

More formal definitions:

- Periodic tasks:
  - Must be executed repeatedly
  - Have an arrival time (beginning of period)
  - Have a deadline (end of period)
- **Aperiodic (sporadic) tasks**:
  - Are executed in response to external event
  - Have an arrival time (when event happens)
  - Have a deadline

Real-time systems

Example

- Periodic tasks (normal work)
- Aperiodic tasks (big red button)



Processing

Somewhere, there is a computer controlling the real-time system

- Sometimes, a PC/laptop
- Most of the times, an embedded computer



Processing

A (single-core) computer

- Can only do 1 thing at a time

Processing

My computer is running:

- PDF viewer
- Email client
- Web browser
- ...
- and it **seems** they all work at the same time

Processing

In (non real-time) computing

- For N tasks
- Executes a bit of task 0, a bit of task 1, 2, ... , N-1
- Then back to the beginning

Round Robin schedule

Processing

In (non real-time) computing, **Round Robin** schedule

- Everything eventually gets executed
- Everything has equal time (regardless of importance)

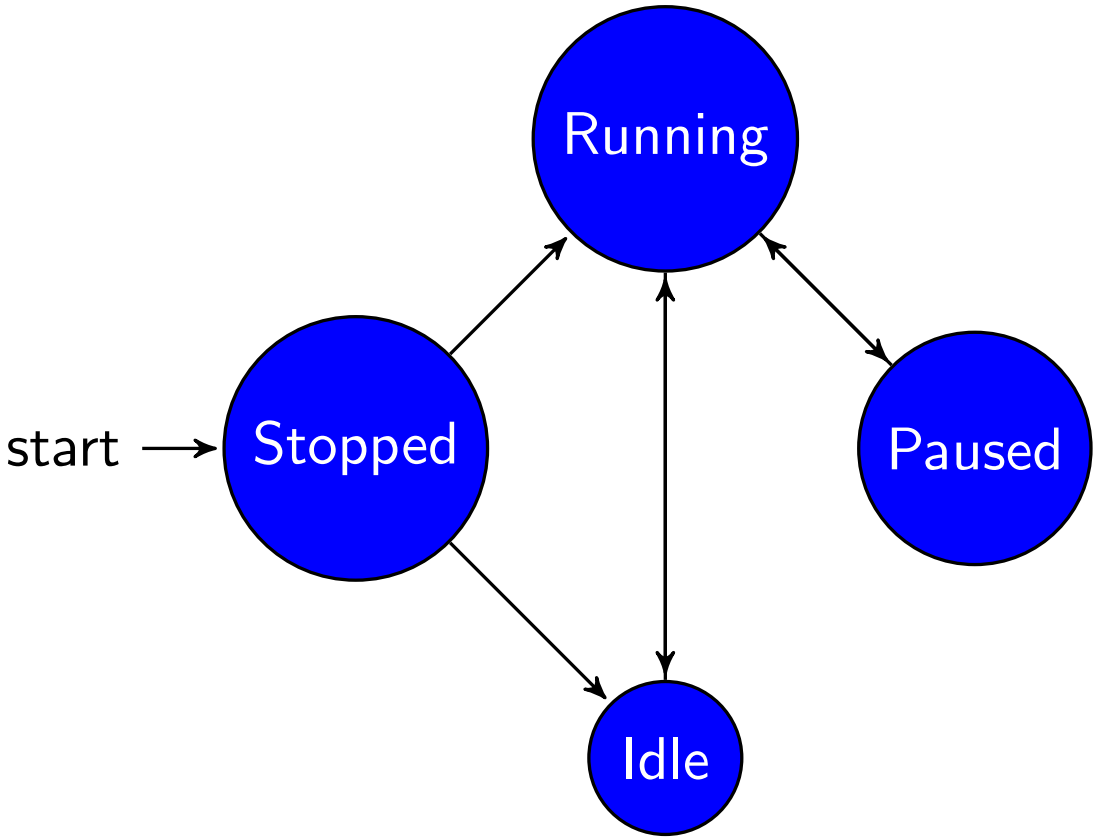
Processing

In real-time computing:

- Round-robin doesn't work
- Let's see how tasks work in a bit more detail

Task states

Task state machine (for each task)



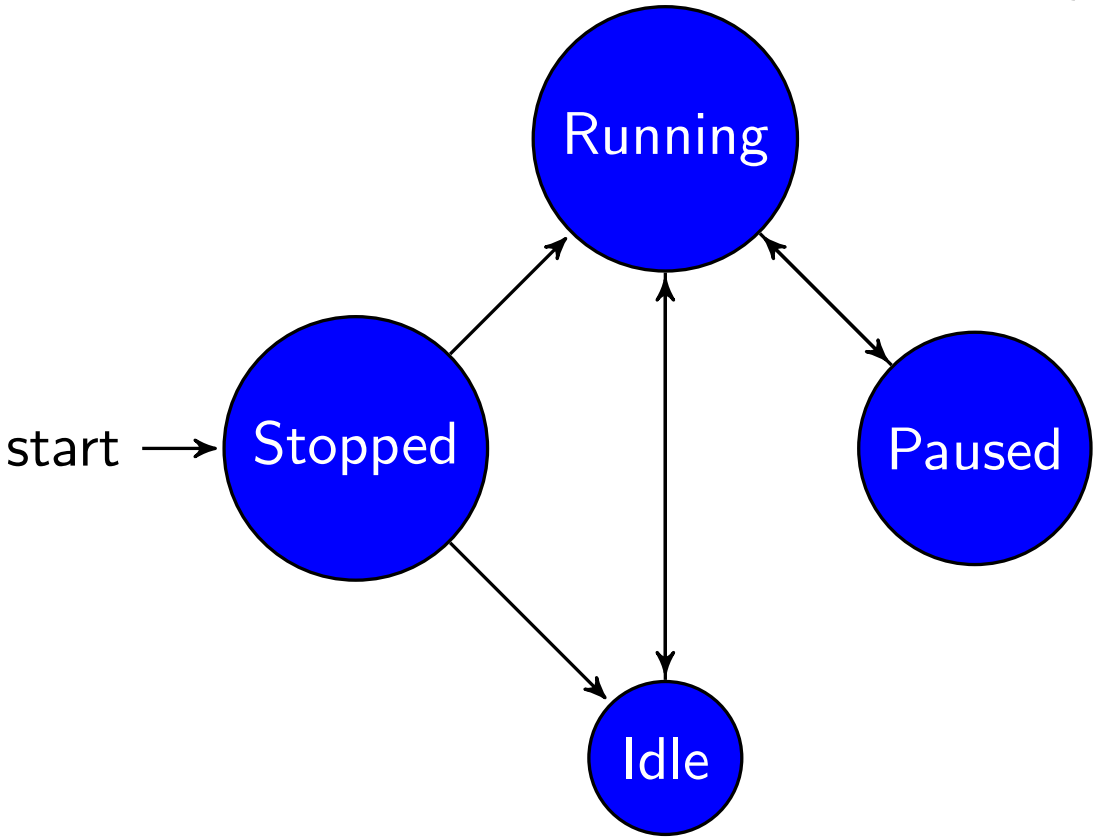
Task states

Stopped

- Task should never execute!
- (because there's no reason for it to run)

Task states

Stopped → Idle: should run, but can't run yet



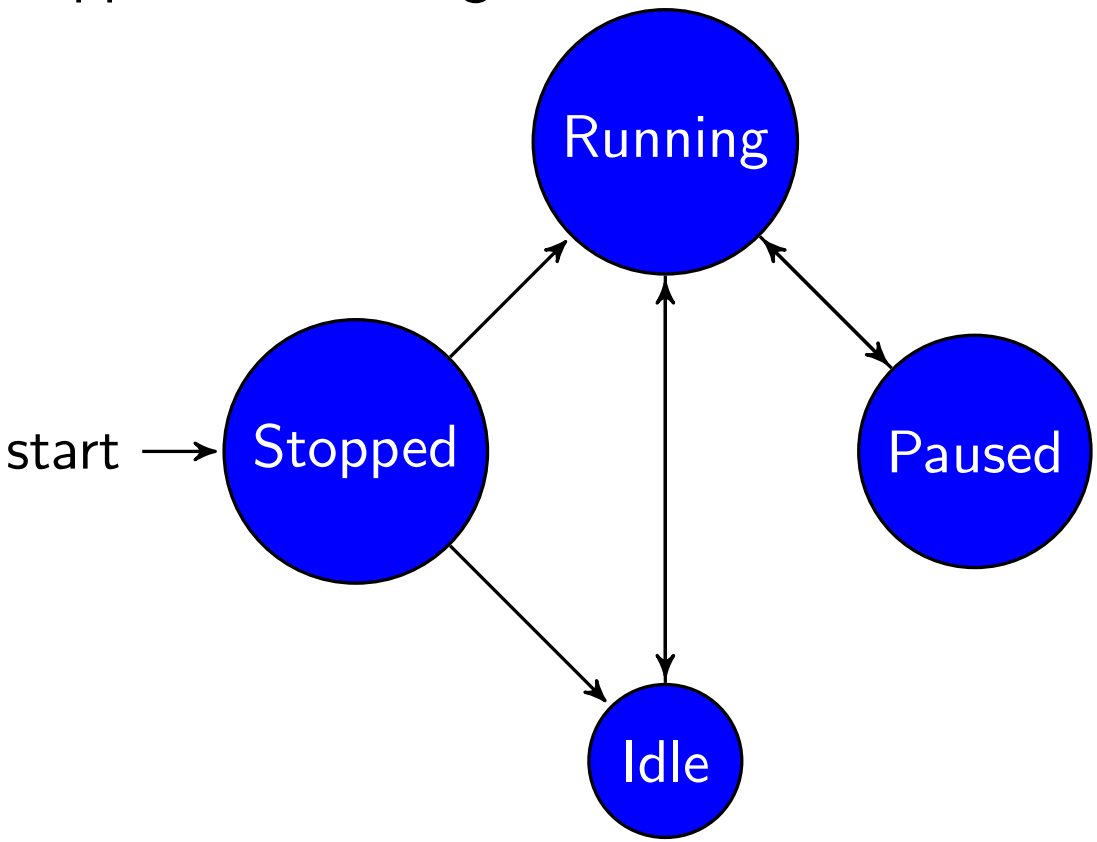
Task states

Idle

- Task should execute
- But is not executing right now, because another task is

Task states

Stopped → Running: should run, and is running



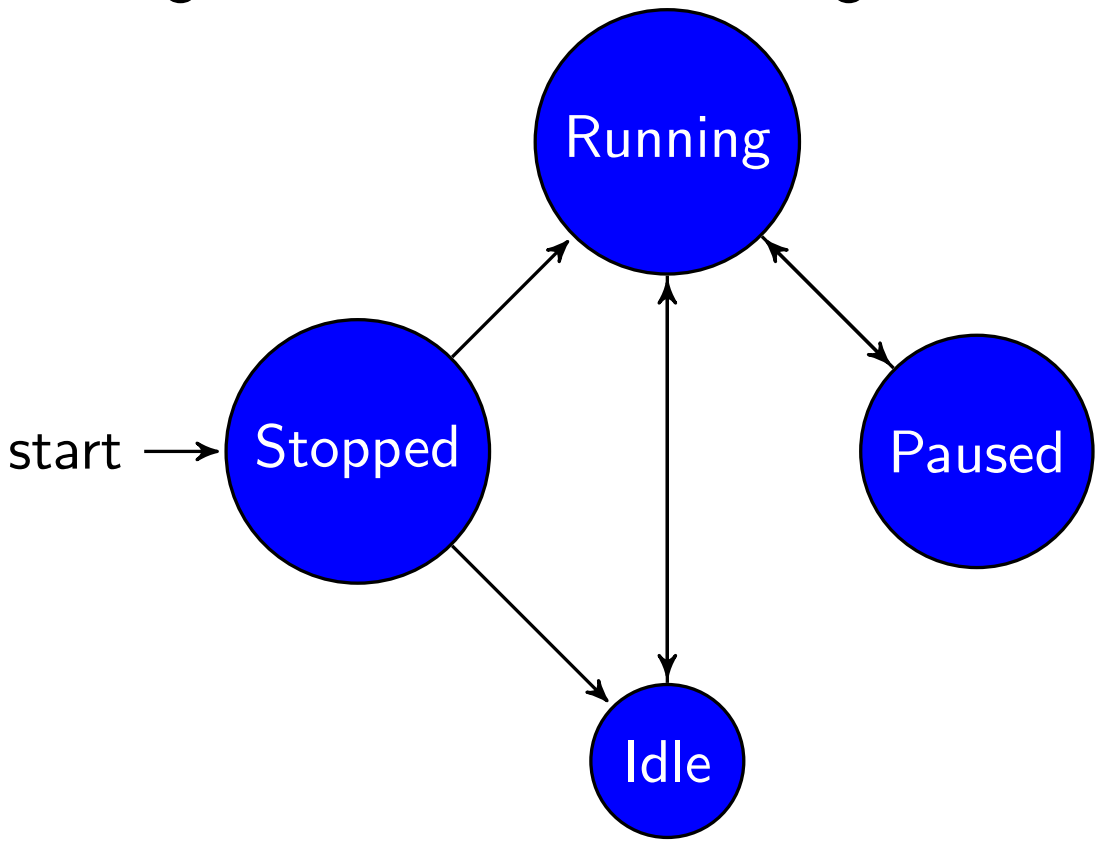
Task states

Running

- Task should execute
- And is executing right now

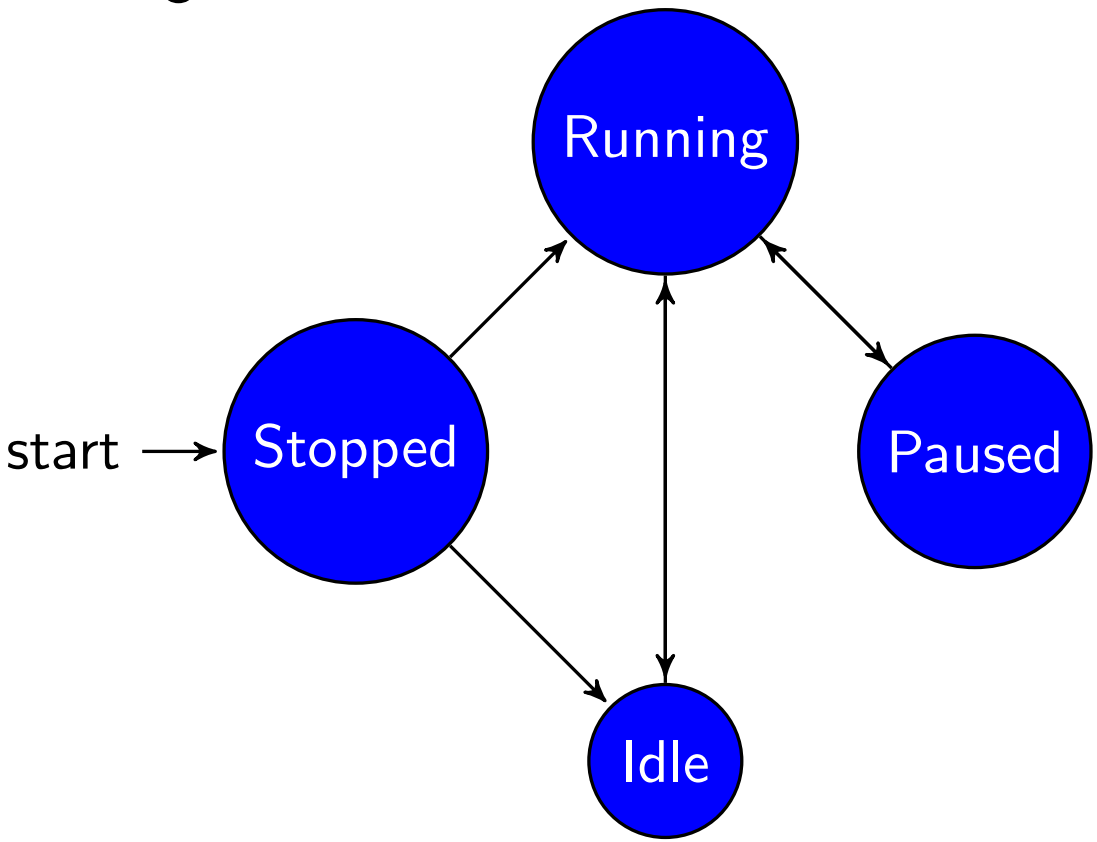
Task states

Running → Idle: finished executing for now



Task states

Running → Paused: hasn't finished, but another task is running



Task states

Paused

- Task started executing, but didn't finish yet
- But is not executing right now, because another task is

Scheduling

(most) systems execute a priority-based schedule

- Each task is assigned a priority (a number)
- (non-stopped) task with the highest priority runs

Scheduling

(most) systems execute a priority-based schedule

- At any given point in time
- Out of all non-stopped tasks
- The one with highest priority is running

Scheduling

Some formality:

- A periodic task is a tuple $\{S, P, E, e\}$

Scheduling

Some formality:

- A periodic task is a tuple $\{S, P, E, e\}$
  - (Tuple just means that you need all these characteristics to define something)

Scheduling

Some formality:

- A periodic task is a tuple $\{S, P, E, e\}$
  - where S is the current state of the task, $S \in \{Stopped, Paused, Idle, Running\}$

Scheduling

Some formality:

- A periodic task is a tuple $\{S, P, E, e\}$
  - where S is the current state of the task, $S \in \{Stopped, Paused, Idle, Running\}$
  - where P is period, $P \in \mathbb{R}$

Scheduling

Some formality:

- A periodic task is a tuple $\{S, P, E, e\}$
  - where S is the current state of the task, $S \in \{Stopped, Paused, Idle, Running\}$
  - where P is period, $P \in \mathbb{R}$
  - where E is total execution time, $E \in \mathbb{R}, E = E_{avg} + \delta E, \delta E \sim \mathcal{N}\left(0, \sigma^2\right)$

Scheduling

Some formality:

- A periodic task is a tuple $\{S, P, E, e\}$
  - where S is the current state of the task, $S \in \{Stopped, Paused, Idle, Running\}$
  - where P is period, $P \in \mathbb{R}$
  - where E is total execution time, $E \in \mathbb{R}, E = E_{avg} + \delta E, \delta E \sim \mathcal{N}\left(0, \sigma^2\right)$
  - where e is completion percentage, $0\% \leq e \leq 100\%$

Real-time Scheduling

Assigning priorities to tasks, such that

- Every task meets its deadline

Real-time Scheduling

- Periodic tasks:
  - Must be executed repeatedly
  - Have an arrival time (beginning of period)
  - Have a deadline (end of period)
- Aperiodic (sporadic) tasks:
  - Are executed in response to external event
  - Have an arrival time (when event happens)
  - Have a deadline

Real-time Periodic Scheduling

Assigning priorities to periodic tasks

- Periodic tasks:
  - Must be executed repeatedly
  - Have an arrival time (beginning of period)
  - Have a deadline (end of period)

so every task meets its deadline

Real-time Periodic Scheduling

If we have

- N tasks
- Each with is own, distinct period and execution time

Is it possible to schedule them, such that all tasks meet their deadline?

Real-time Periodic Scheduling

Yes, if and only if

- $U = \sum_{i=1}^{n} U_i = \sum_{i=1}^{n} \frac{E_i}{P_i} \leq n(2^{1/n} - 1)$

A result proven by Liu and Layland in 1973[1]

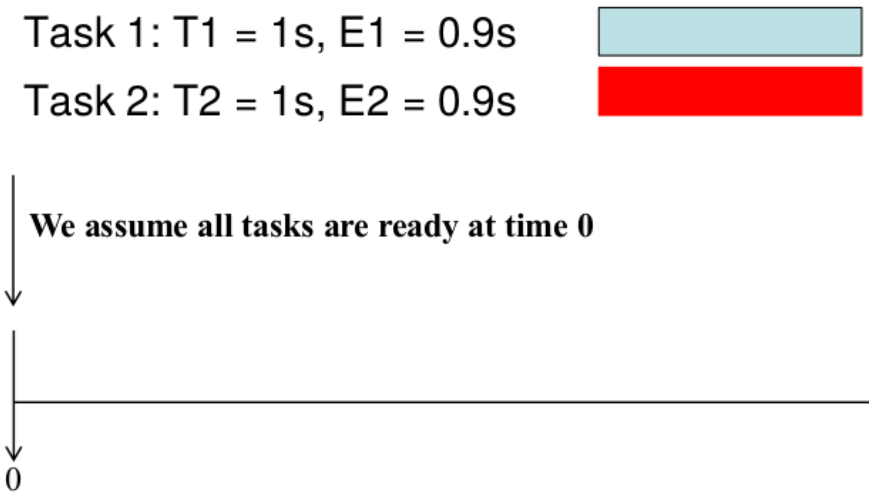
[1] Liu, C. L.; Layland, J. (1973), "Scheduling algorithms for multiprogramming in a hard real-time environment", Journal of the ACM, 20 (1): 46–61

Real-time Periodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s  P = 2
  - Task 2: T2 = 1s, E2 = 0.9s  P = 1

Real-time Periodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



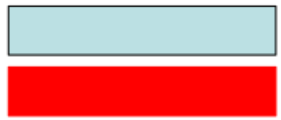
Real-time Periodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



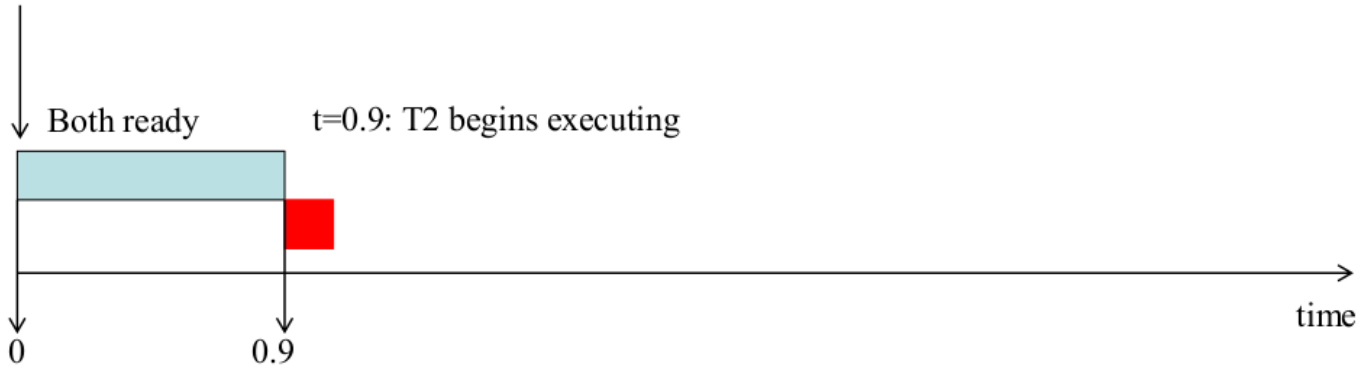
Real-time Periodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



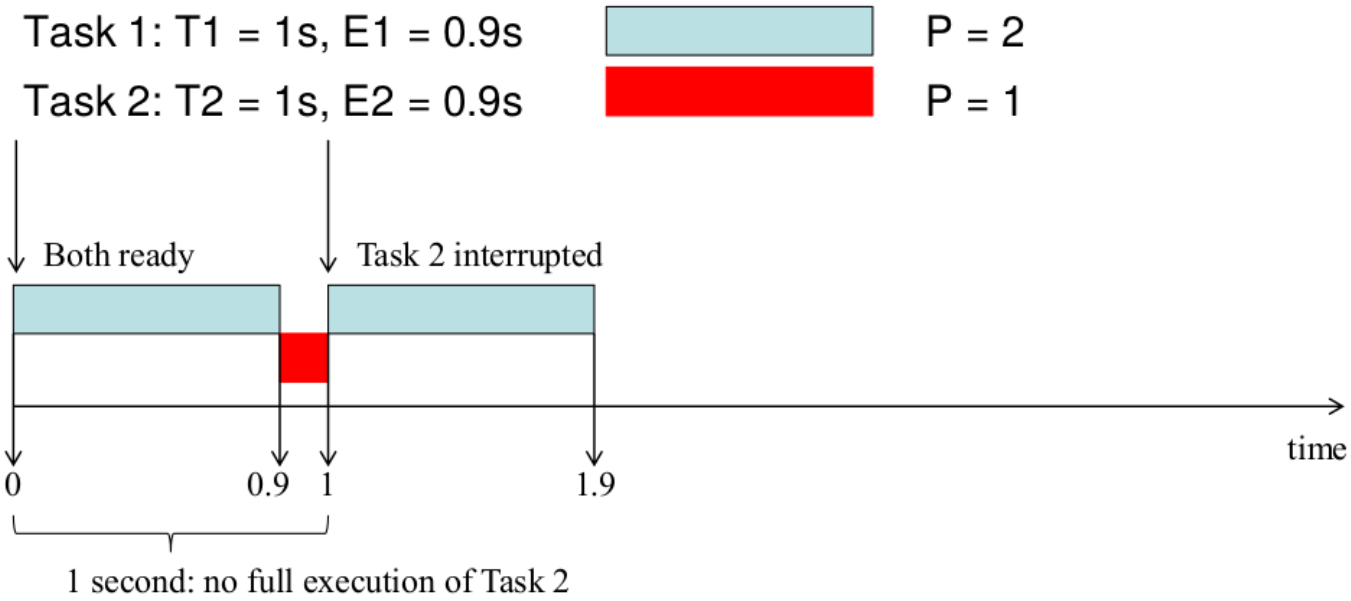
P = 2

P = 1



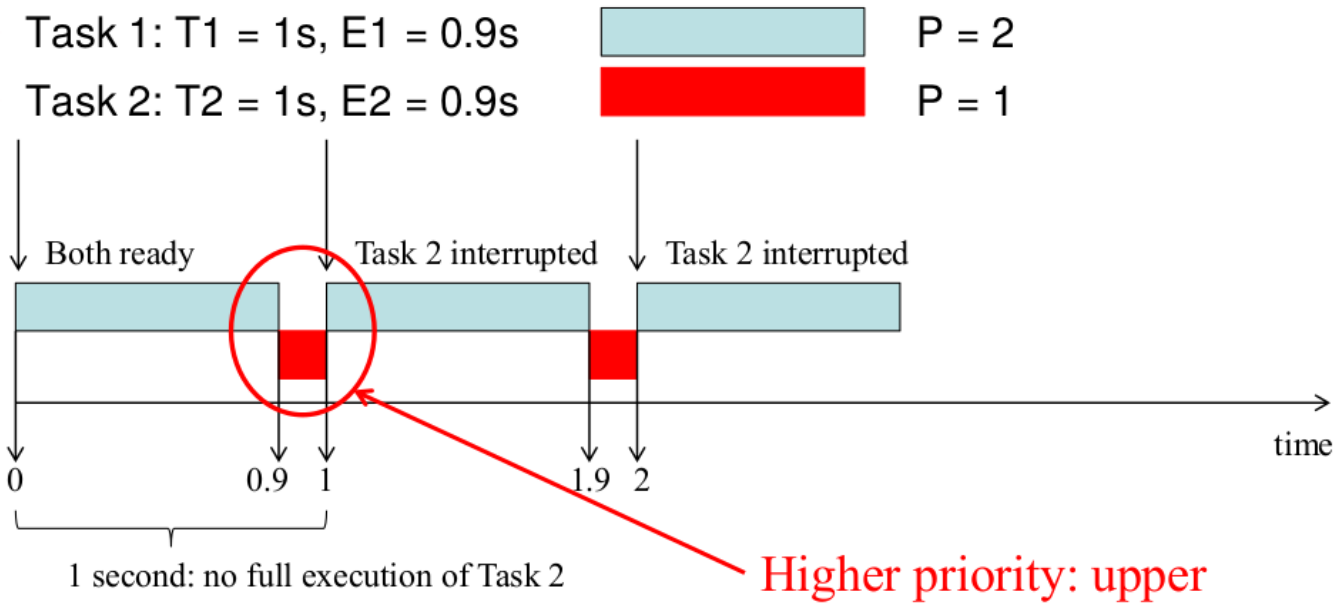
Real-time Periodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



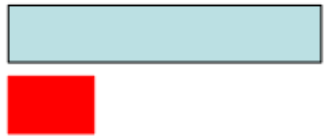
Real-time Periodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



Real-time Periodic Scheduling

- A schedulable example (badly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s

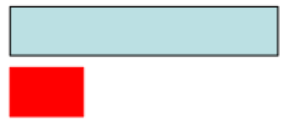


P = 2

P = 1

Real-time Periodic Scheduling

- A schedulable example (badly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



P = 2

P = 2

P = 1



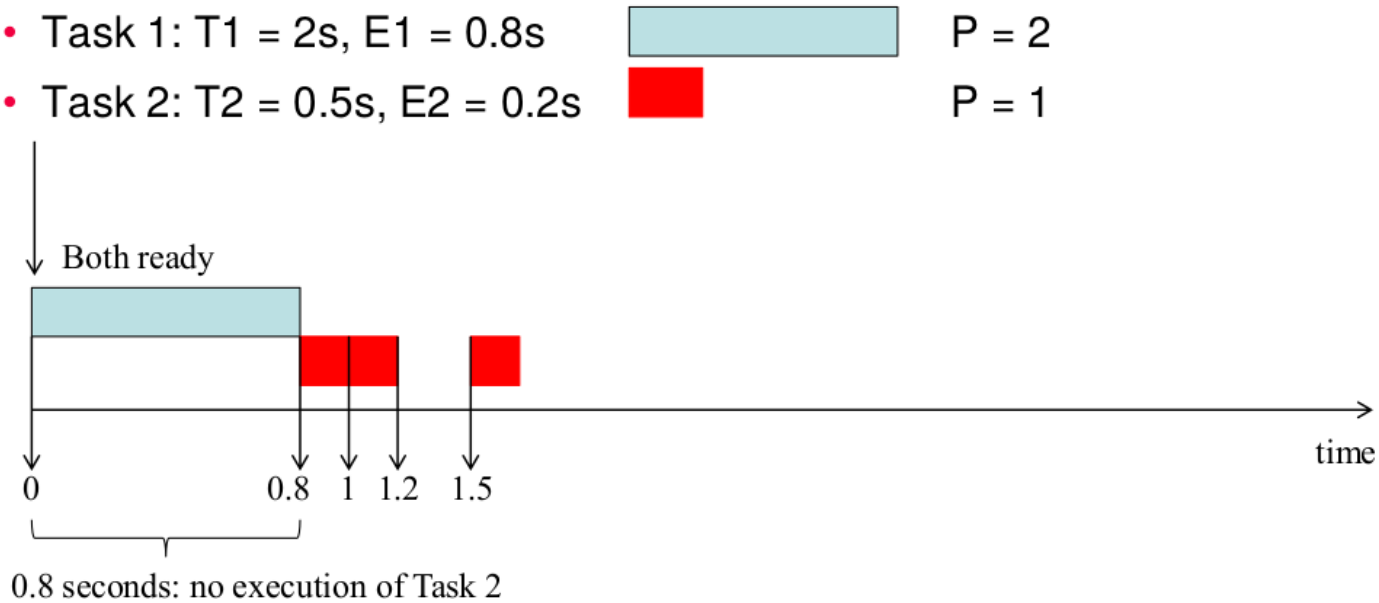
Real-time Periodic Scheduling

- A schedulable example (badly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



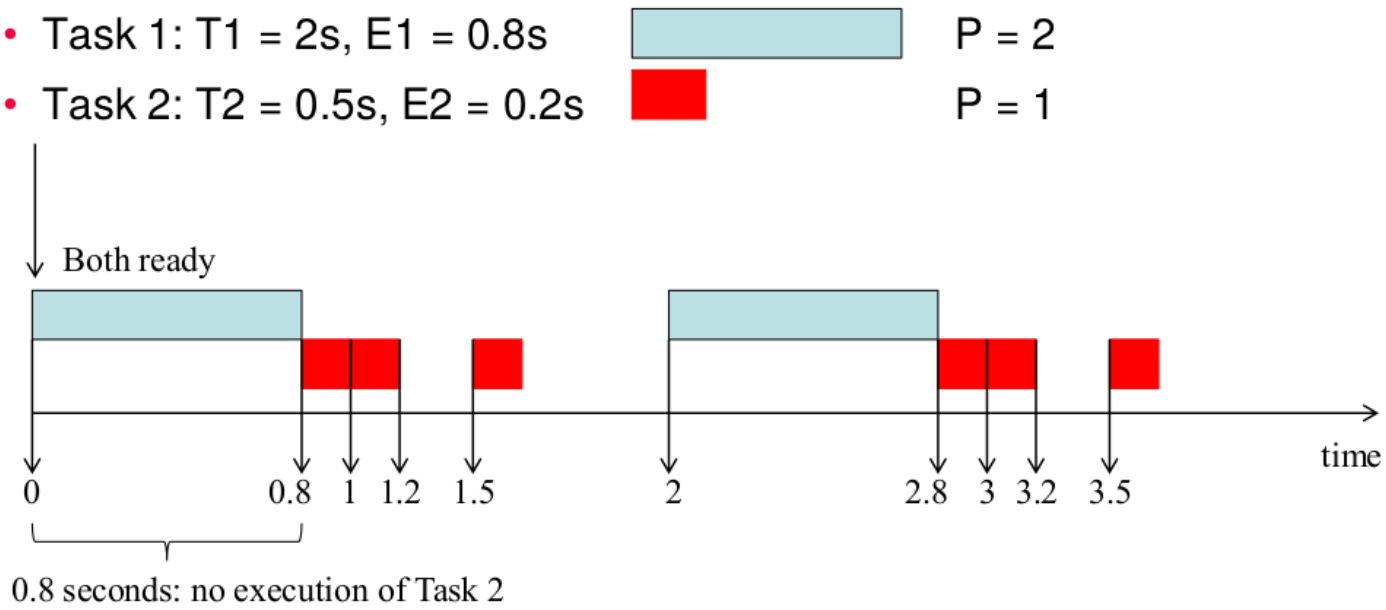
Real-time Periodic Scheduling

- A schedulable example (badly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



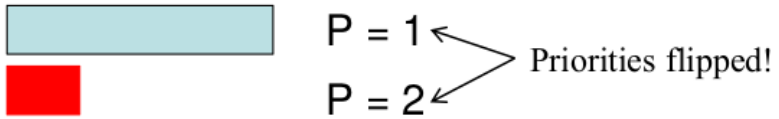
Real-time Periodic Scheduling

- A schedulable example (badly implemented)



Real-time Periodic Scheduling

- A schedulable example (correctly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



Real-time Periodic Scheduling

- A schedulable example (correctly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



Real-time Periodic Scheduling

- A schedulable example (correctly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



Real-time Periodic Scheduling

- A schedulable example (correctly implemented)



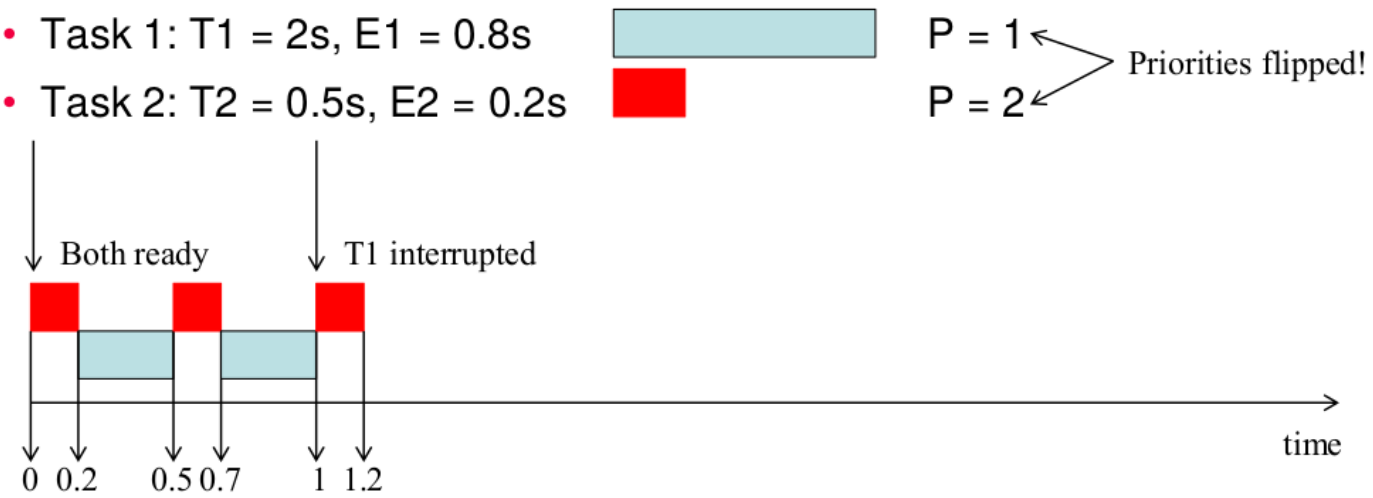
Real-time Periodic Scheduling

- A schedulable example (correctly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



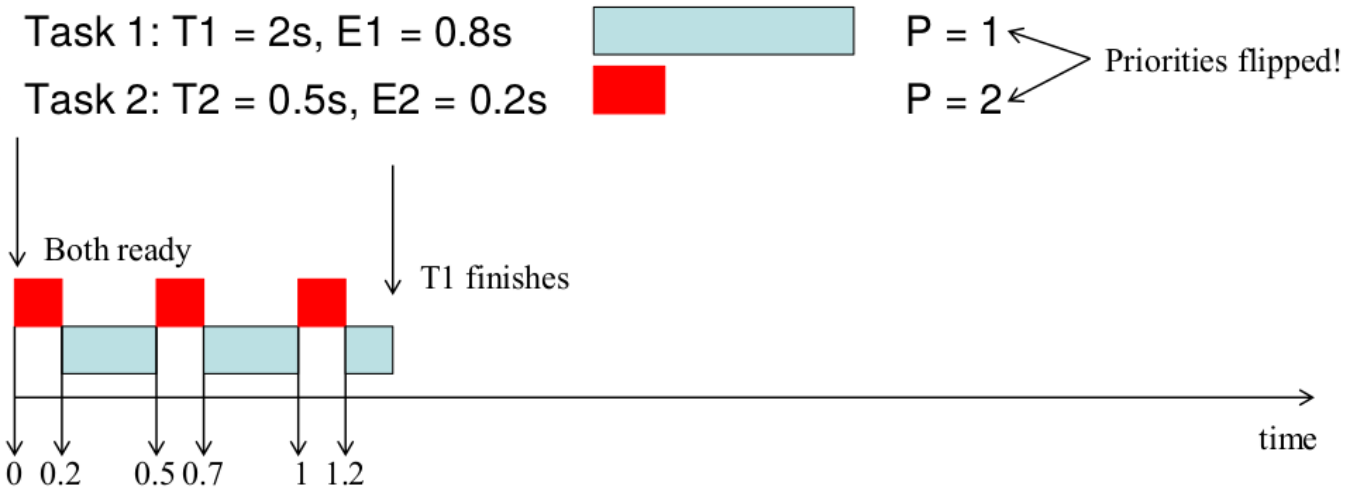
Real-time Periodic Scheduling

- A schedulable example (correctly implemented)



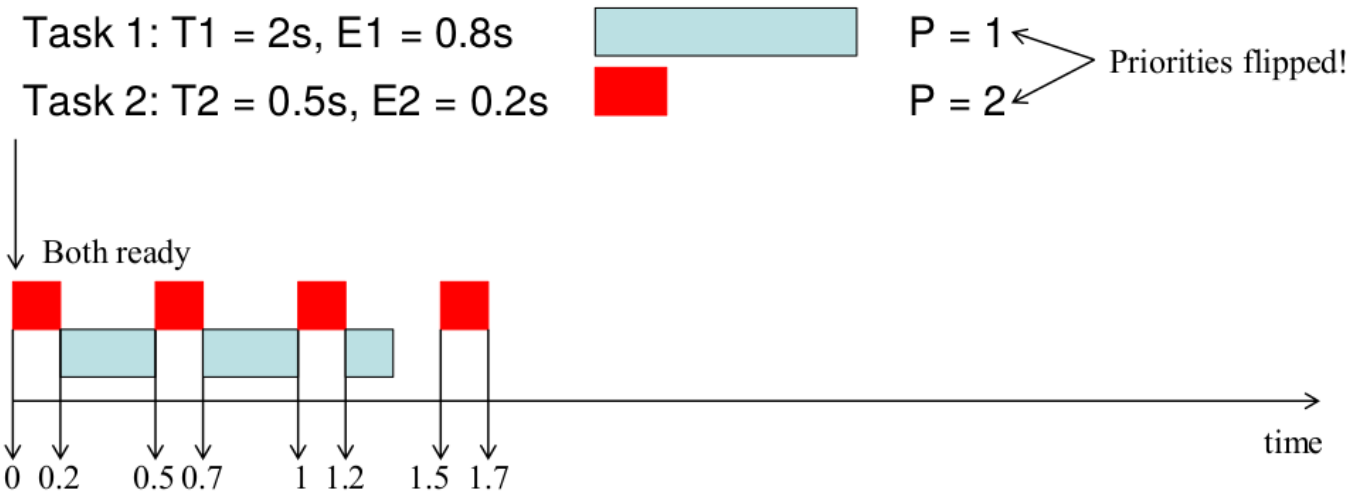
Real-time Periodic Scheduling

- A schedulable example (correctly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



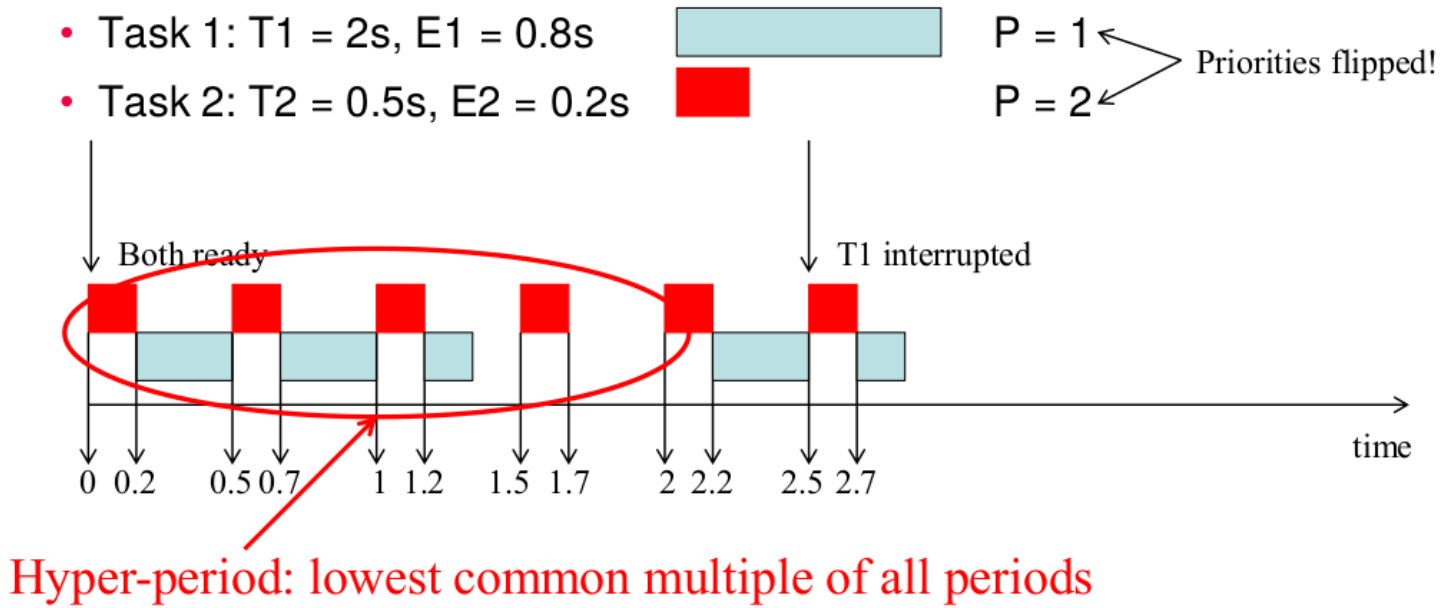
Real-time Periodic Scheduling

- A schedulable example (correctly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s



Real-time Periodic Scheduling

- A schedulable example (correctly implemented)



Real-time Periodic Scheduling

What we just did is called **RMS**

- Rate-Monotonic Scheduling
  - Prorities assigned in inverse proportion to period
  - (shortest period gets highest priority)

Real-time Periodic Scheduling

RMS is optimal

- If a task set is schedulable...
  - RMS is guaranteed to schedule it
  - (for periodic tasks only)

Real-time Periodic Scheduling

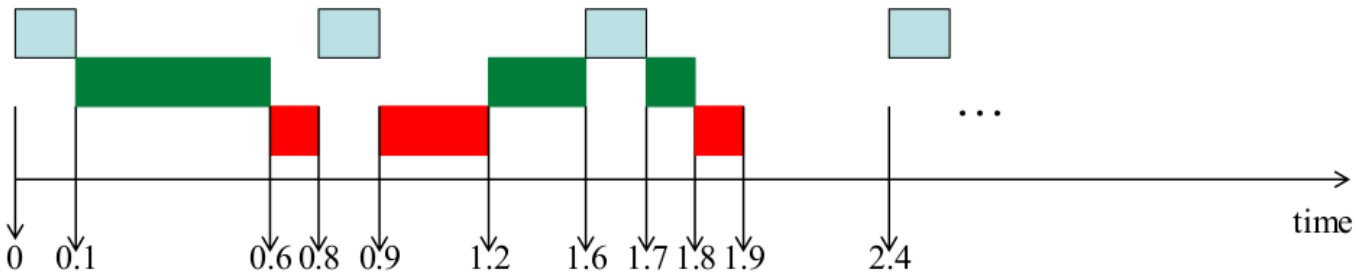
- Exercise
  - Task 1: T=0.8, E=0.1
  - Task 2: T=2.4, E=0.6
  - Task 3: T=1.2, E=0.5



  - Assign priorities according to RMS
  - Draw one hyper-period of the task scheduling diagram

Real-time Periodic Scheduling

- Exercise
  - Task 1: T=0.8, E=0.1 P=3
  - Task 2: T=2.4, E=0.6 P=1
  - Task 3: T=1.2, E=0.5 P=2



Discussion

- We made some implicit assumptions (that affect the outcome)

Discussion

- We made some implicit assumptions (that affect the outcome)
  - We modeled execution time as constant

Discussion

- We made some implicit assumptions (that affect the outcome)
  - We modeled execution time as constant
  - We modeled task-switching time as 0

Discussion

- We made some implicit assumptions (that affect the outcome)
  - We modeled execution time as constant
  - We modeled task-switching time as 0
  - We assumed taks can be paused

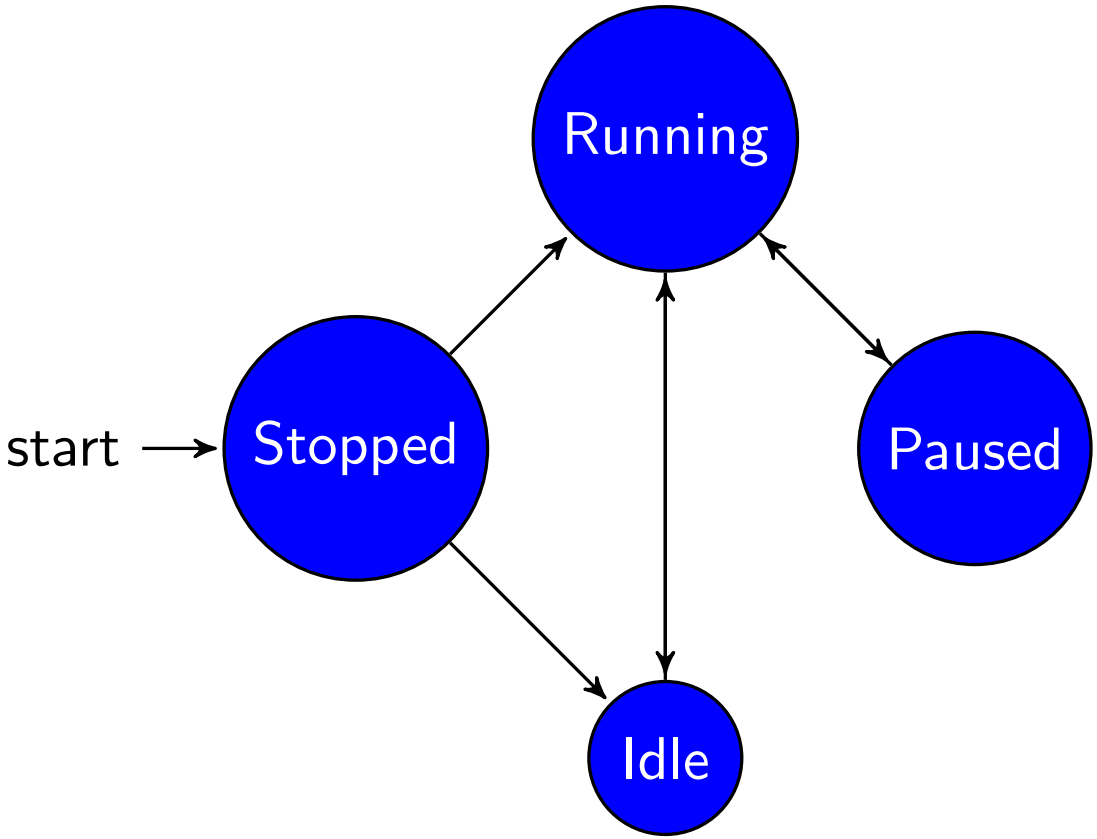
Scheduling

Aperiodic scheduling

- Dynamic scheduling
- For periodic and aperiodic tasks

Task states

Task state machine (for each task)



Scheduling

Aperiodic tasks

- Are responses to external events
- Something happens: task must execute

Scheduling

Aperiodic tasks have:

- an arrival time **A**
- an execution time **E**
- a deadline **D** (relative to A)

Aperiodic Scheduling

Earliest-Deadline First (EDF)

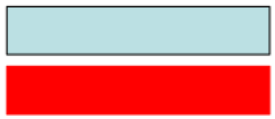
- the active task with the earliest deadline executes
- (priorities are assigned, dynamically, in inverse proportion to deadlines)

Aperiodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s  P = 2
  - Task 2: T2 = 1s, E2 = 0.9s  P = 1

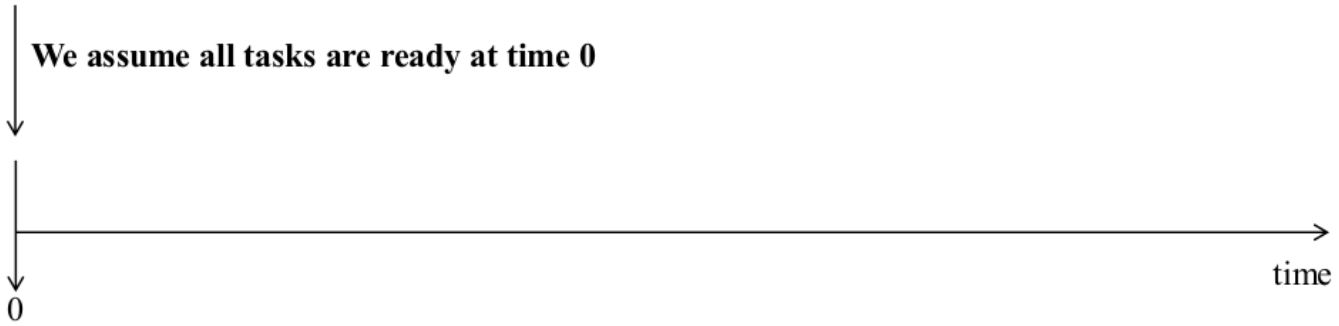
Aperiodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



P = 2

P = 1



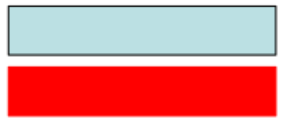
Aperiodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



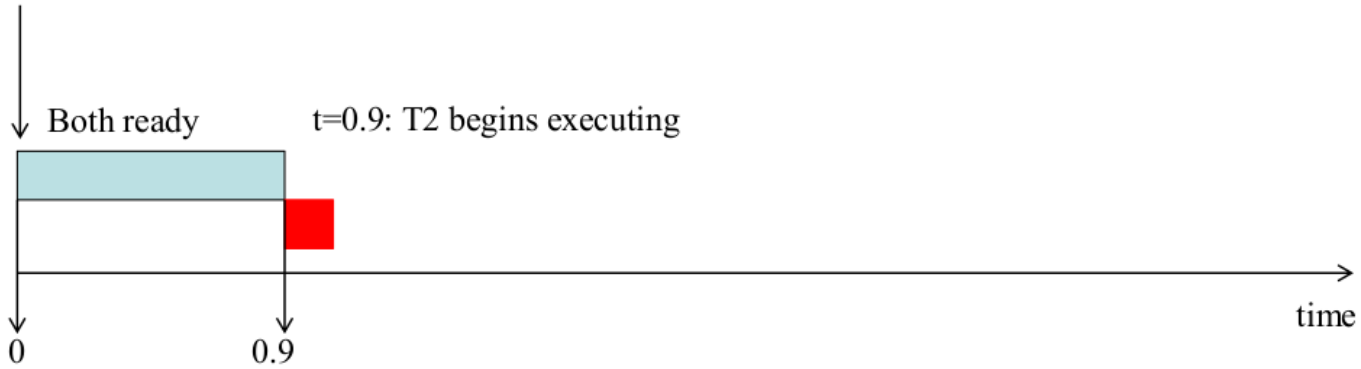
Aperiodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



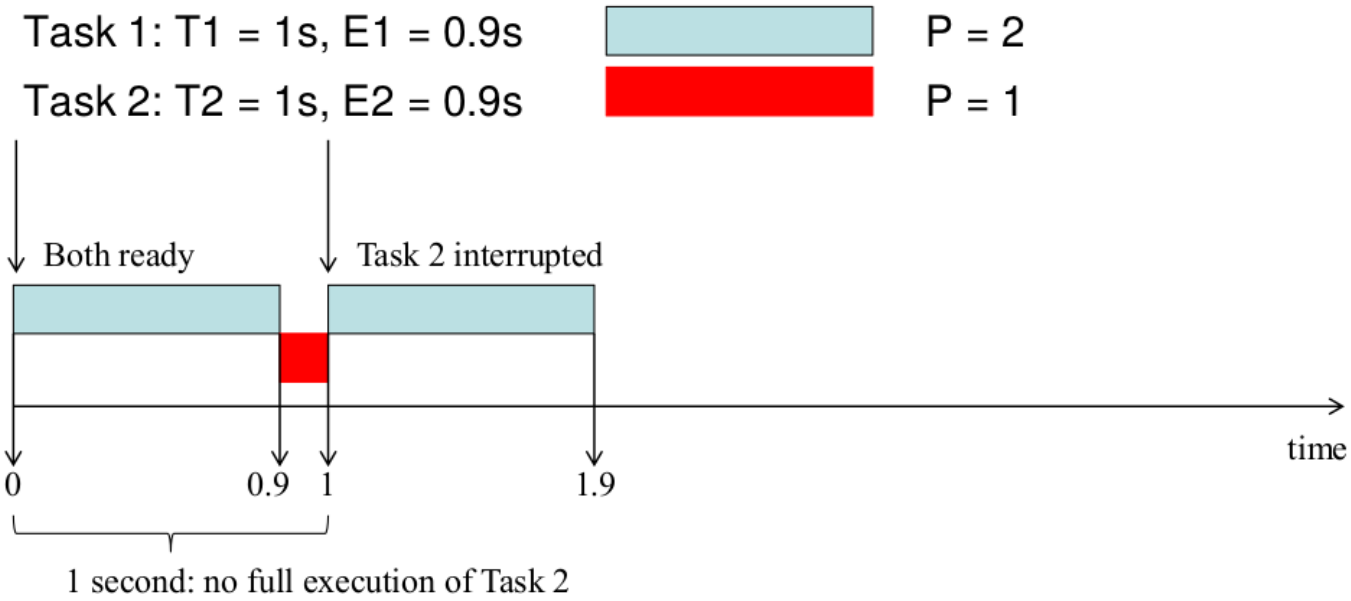
P = 2

P = 1



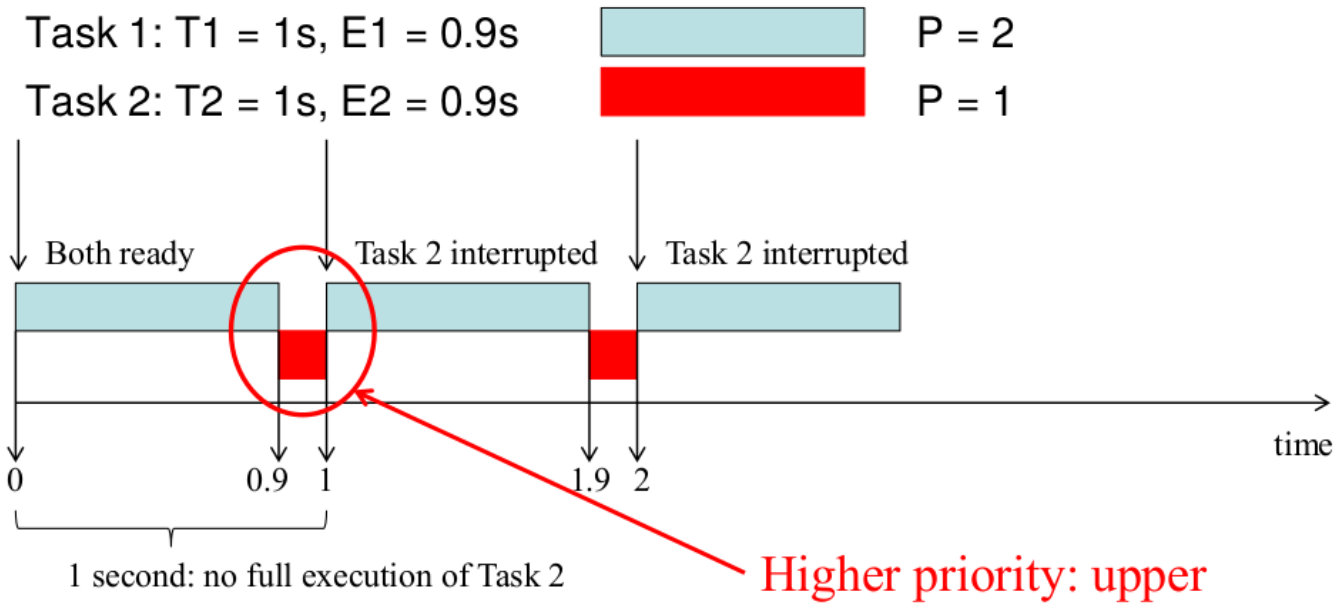
Aperiodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



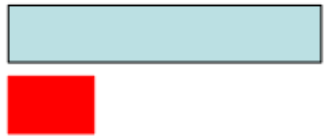
Aperiodic Scheduling

- A non-schedulable example
  - Task 1: T1 = 1s, E1 = 0.9s
  - Task 2: T2 = 1s, E2 = 0.9s



Aperiodic Scheduling

- A schedulable example (badly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s
  - Task 2: T2 = 0.5s, E2 = 0.2s

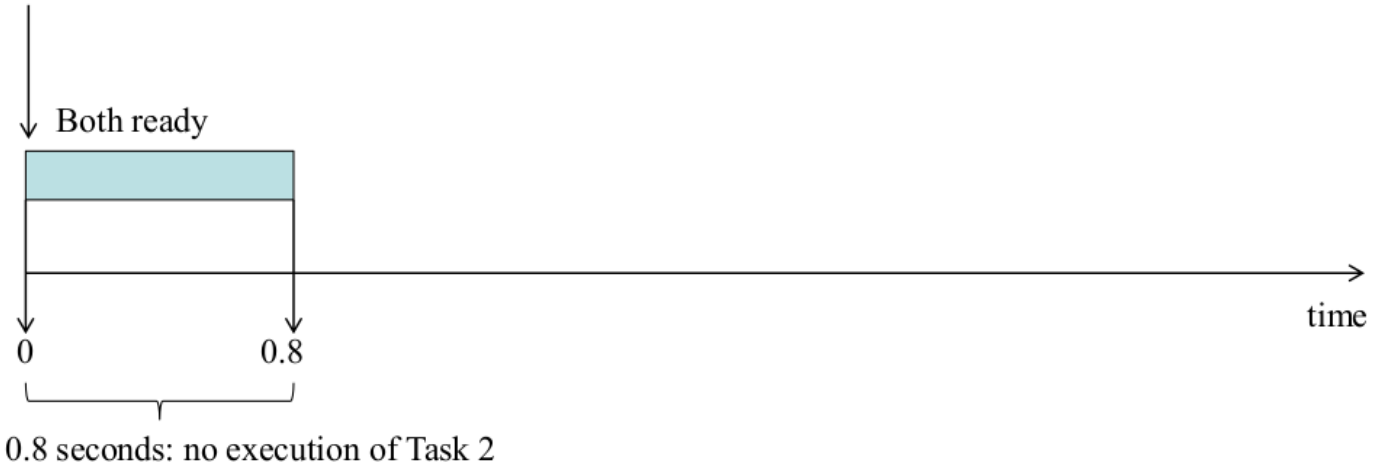


P = 2

P = 1

Aperiodic Scheduling

- A schedulable example (badly implemented)
  - Task 1: T1 = 2s, E1 = 0.8s  P = 2
  - Task 2: T2 = 0.5s, E2 = 0.2s  P = 1



Aperiodic Scheduling

EDF is not the only algorithm

- e.g., Least Slack-Time First (LSF)
- EDF is optimal: if something can be scheduled, EDF will schedule it

Aperiodic Scheduling

Critical idea:

- We must always look at the worst-case scenario
- And ensure worst-case is schedulable

Scheduling

Scheduling is (relatively) easy if:

- Tasks are independent of each other

Scheduling

A task is independent of others if:

- It can run, regardless of what other tasks are running

Scheduling

A task is independent of others if:

- It can run, regardless of what other tasks are running
- (in practice, we're never this lucky)

Scheduling

Some nuanced distinctions

- Parallelism: things happen at the same time, but they're fairly synchronized
- Concurrency: things happen at the same time, but there's no synchronization

Race conditions

- Let's look at some code

Race conditions

Problem was a race condition

- **when** something happens affects correctness

Race conditions

Race conditions occur when:

- Parallel tasks **share** resources
- There is no control on resource access

Race conditions

Solution

- Not share resource (sometimes not viable)
- Control access: ensure right order is preserved
- (code example: solution)

Mutexes

Gatekeepers

- Only the task that holds the mutex can do something
- All others shall not pass

Mutexes

Mutex

- **Mut**ual **ex**clusion

Mutexes

Any process that is about to enter a **critical region**

- A part of the process that uses shared resources

Mutexes

Any process that is about to enter a **critical region**

- Must wait for mutex to become available

Mutexes

Any process that is about to enter a **critical region**

- Must wait for mutex to become available
- Acquire the mutex

Mutexes

Perform **critical region** work

- Release the mutex

Mutexes

Race conditions averted

- But...
- .. what if 2 processes try to acquire mutex at same time?

Mutexes

Implementation

- Hardware atomic operations (e.g., swap)
- Software algorithms in OS/library code

Mutexes

Notice the critical thing

- Mutexes solve a concurrency problem

Mutexes

Notice the critical thing

- Mutexes solve a concurrency problem
- By assuming **the problem is solved at a lower level**

Mutexes

This isn't always true

- Particularly in the context of robotics
- We'll later see a more general solution (PAXOS)

Mutexes

Recap:

- Concurrency causes race conditions
- Mutexes can be used to solve that problem
  - Concurrency requires synchronization mechanisms

Mutexes

Race conditions averted

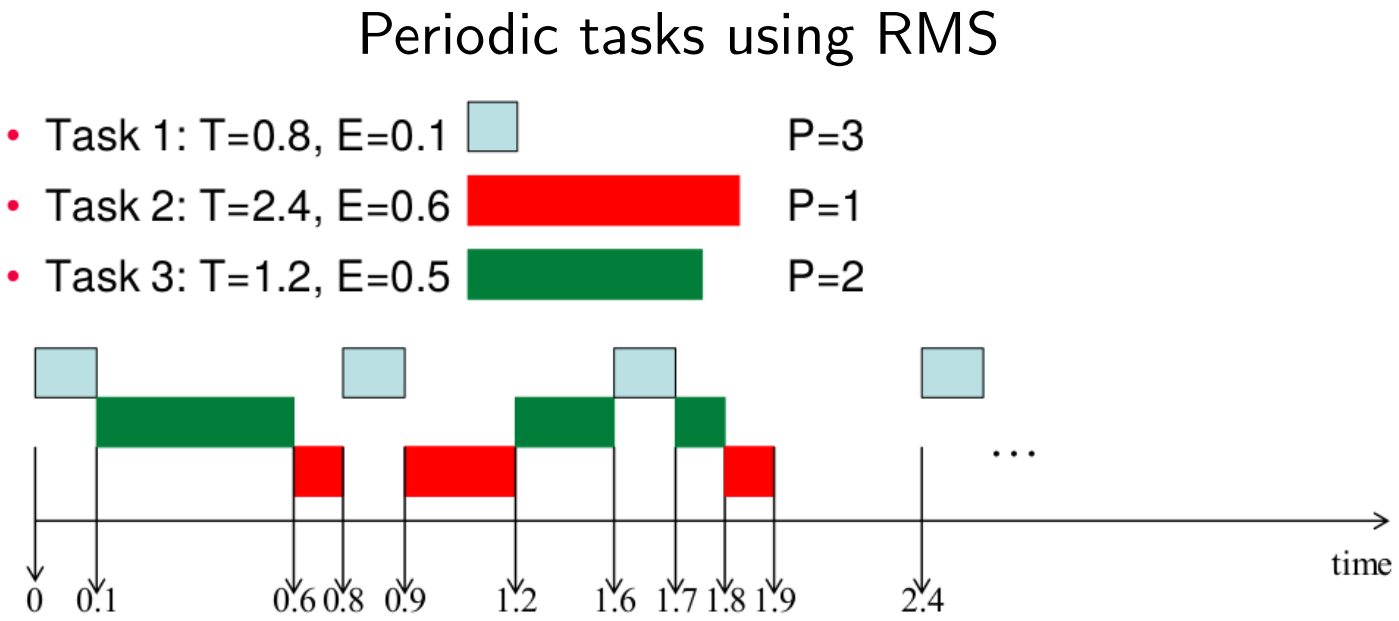
- But...
- .. we now have resource contention (competing to enter critical region)

Priority inversion

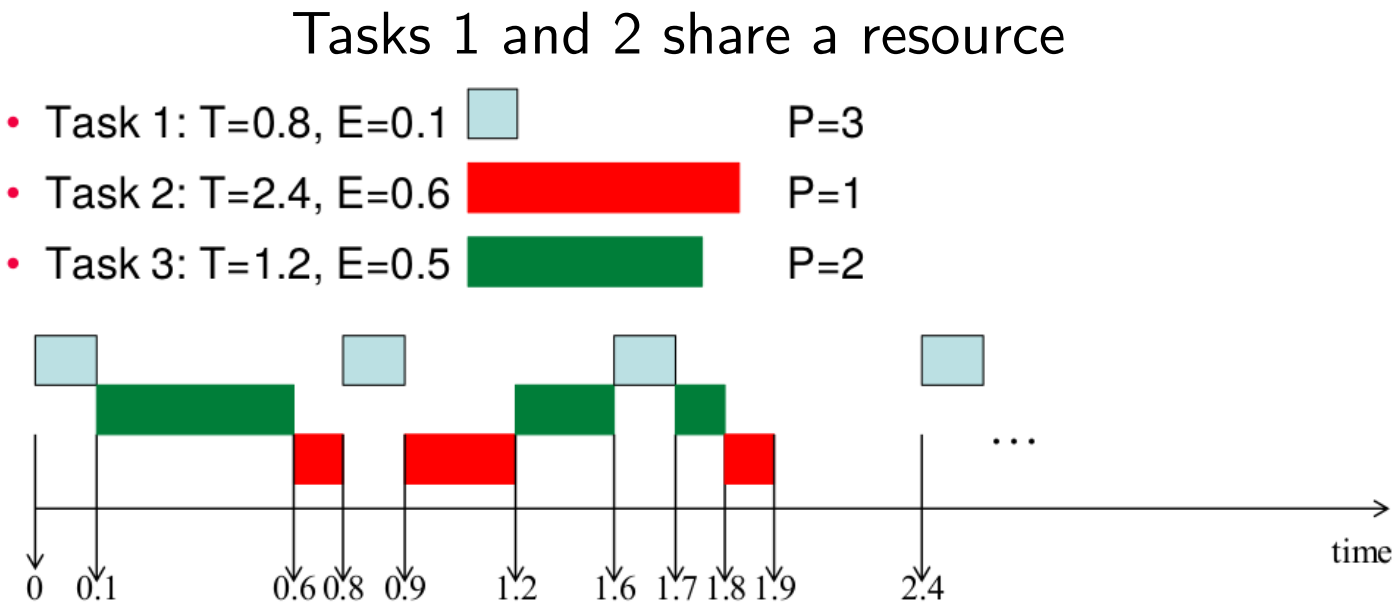
In real-time systems

- Resource contention potentially causes priority inversion

Priority inversion



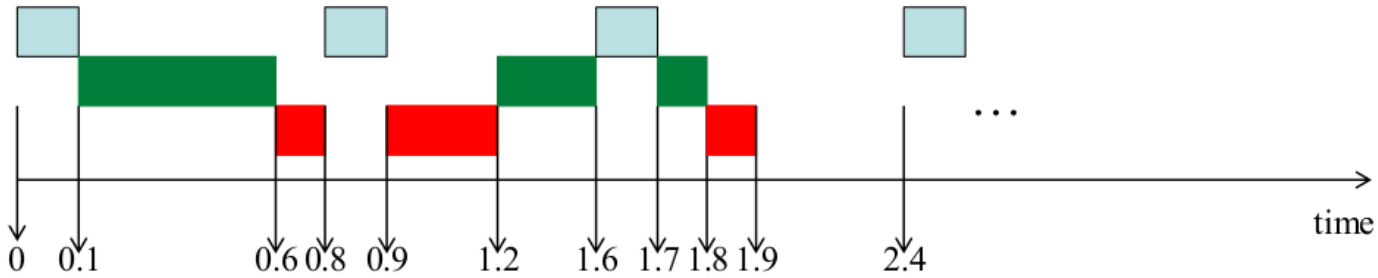
Priority inversion



Priority inversion

Each tries to acquire mutex at beginning of execution

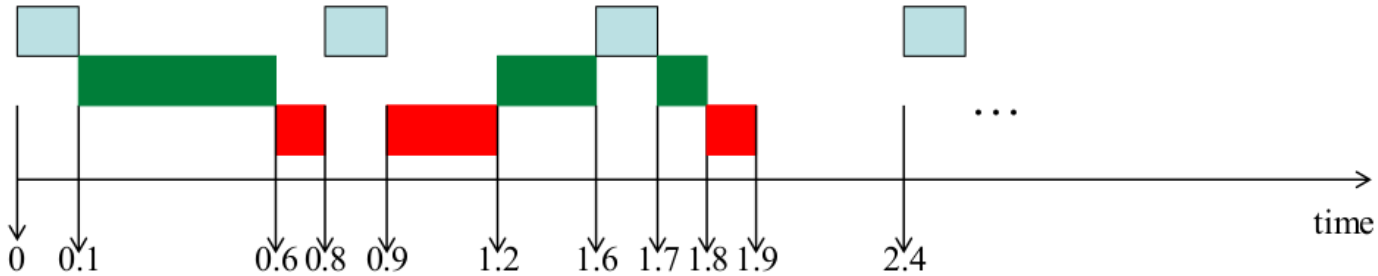
- Task 1: T=0.8, E=0.1 P=3
- Task 2: T=2.4, E=0.6 P=1
- Task 3: T=1.2, E=0.5 P=2



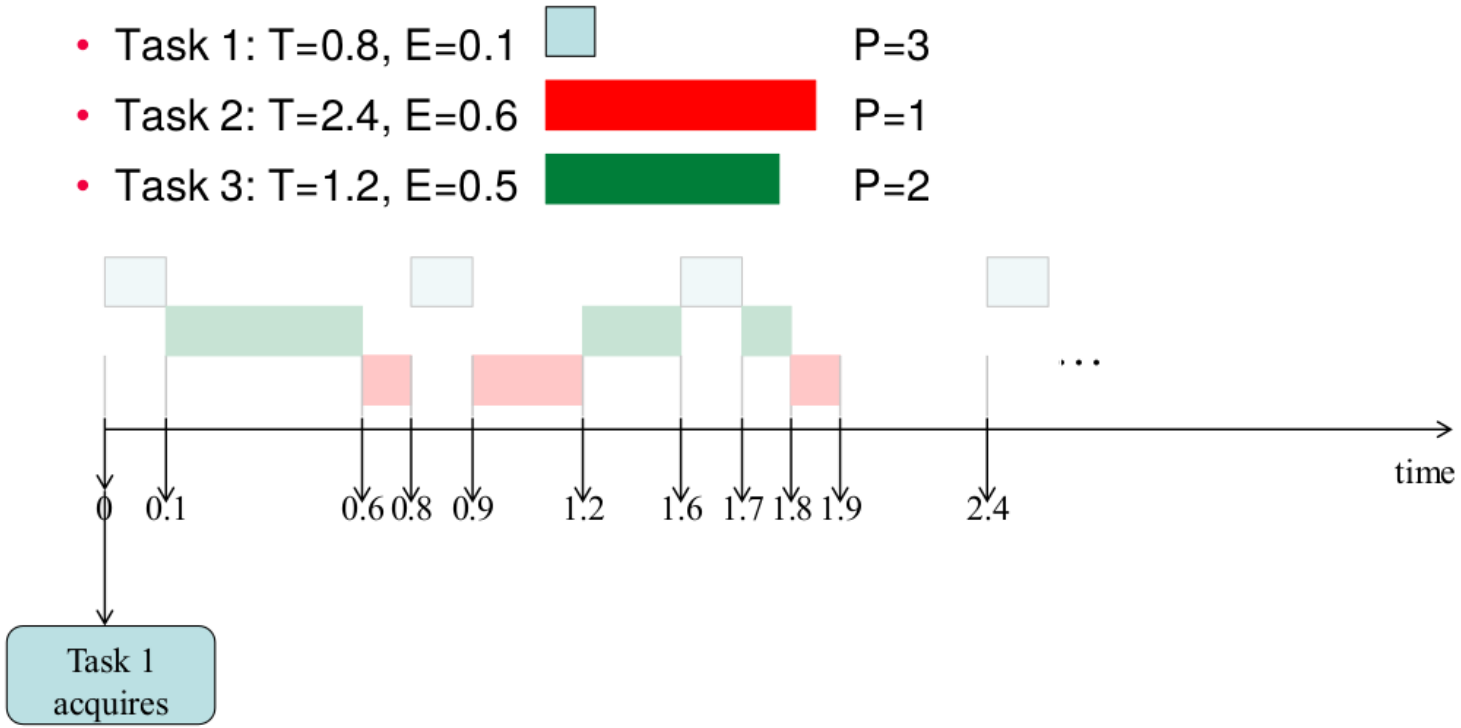
Priority inversion

Each releases mutex at end of execution

- Task 1: T=0.8, E=0.1 P=3
- Task 2: T=2.4, E=0.6 P=1
- Task 3: T=1.2, E=0.5 P=2

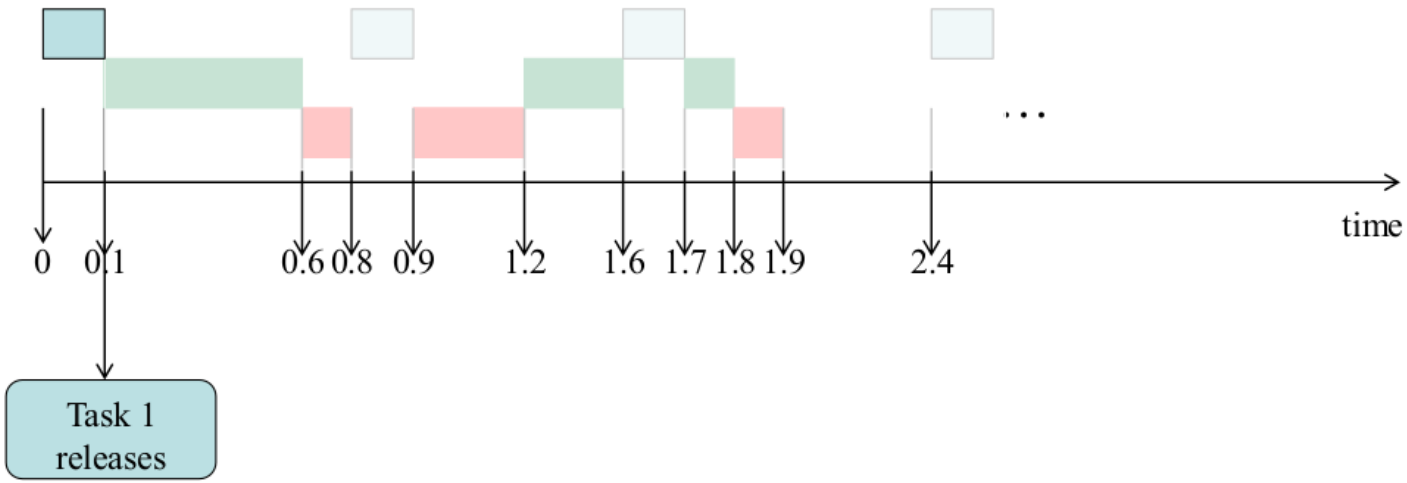


Priority inversion



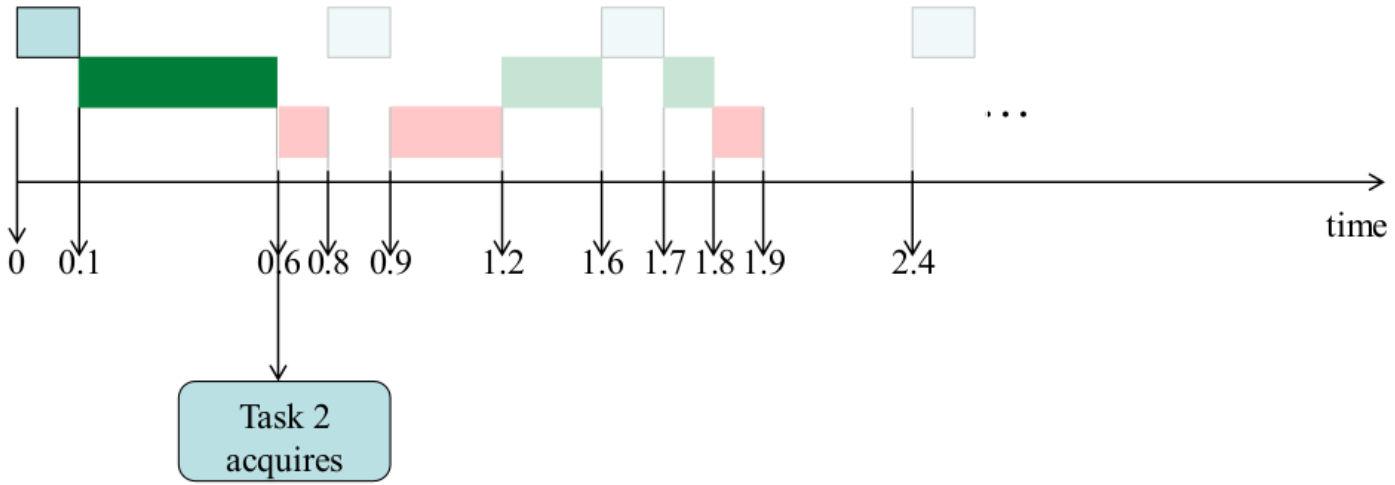
Priority inversion

- Task 1: T=0.8, E=0.1 P=3
- Task 2: T=2.4, E=0.6 P=1
- Task 3: T=1.2, E=0.5 P=2

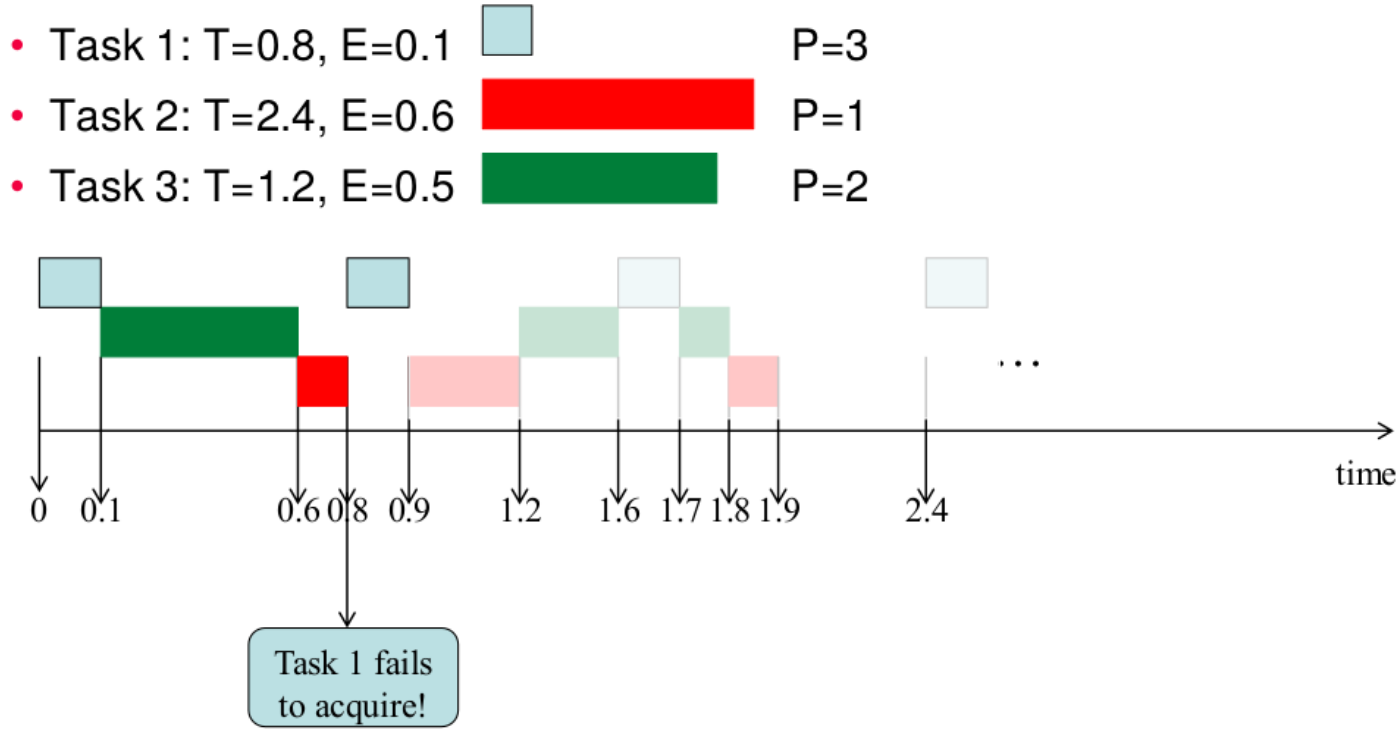


Priority inversion

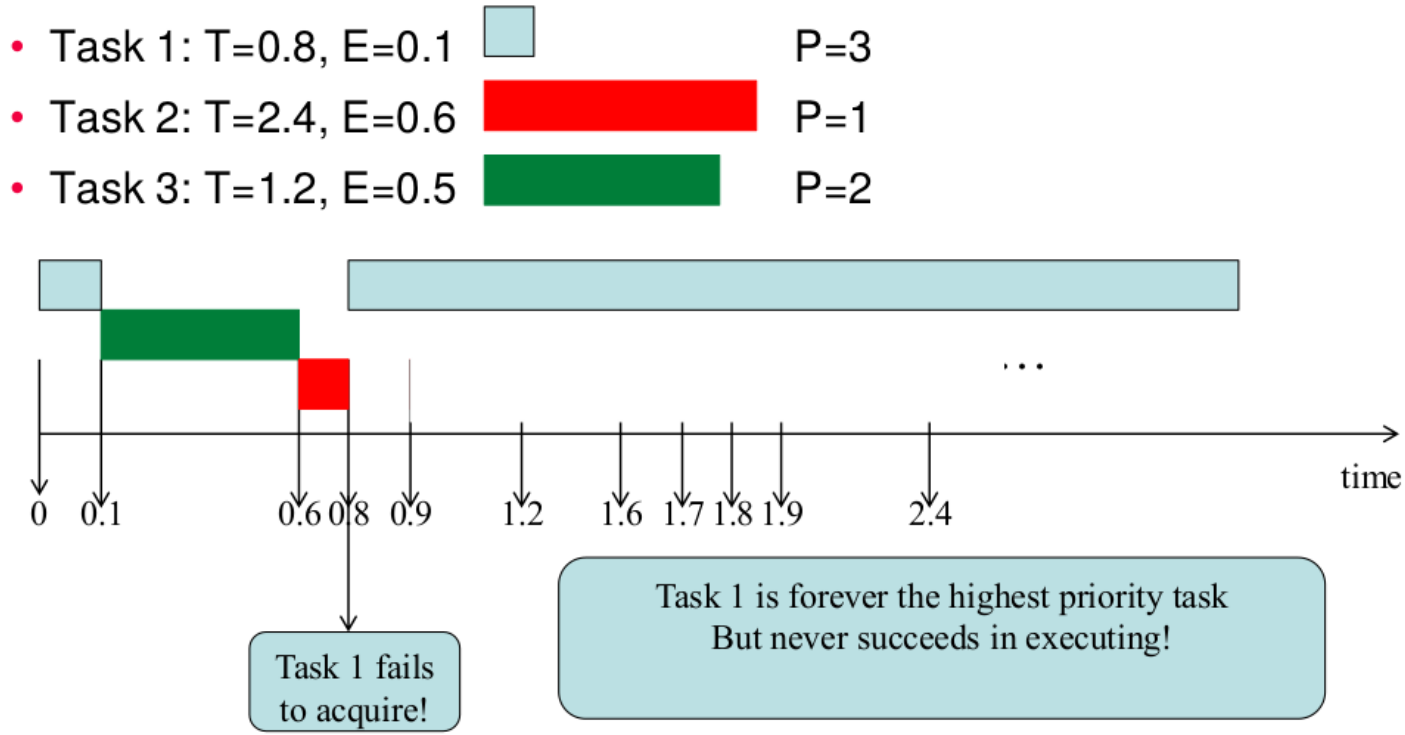
- Task 1: T=0.8, E=0.1 P=3
- Task 2: T=2.4, E=0.6 P=1
- Task 3: T=1.2, E=0.5 P=2



Priority inversion



Priority inversion



Priority inversion

What happened?

- Our scheduling policy (RMS) didn't account for failure to acquire resource
- Coupled with resource contention, we have a deadlock!
- Solution (?): block tasks that fail to acquire mutex

Priority inversion

Case study: Mars Pathfinder

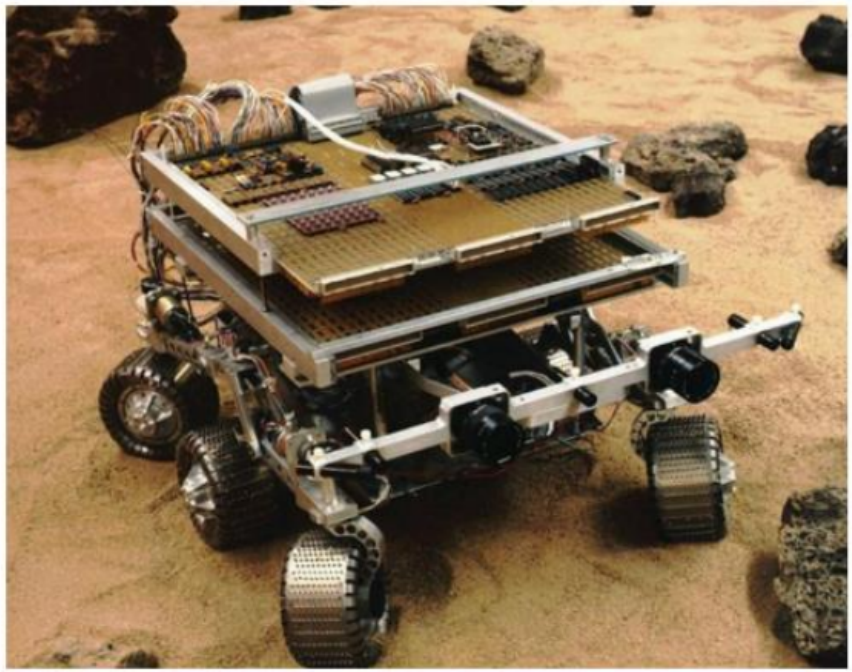
- First mission to land a rover on Mars
  - Dec. 4, 1996
    - Pathfinder launched.
  - July 4, 1997
    - Pathfinder lands on Mars
  - Sept. 27, 1997
    - last successful data transmission from Mars Pathfinder



Priority inversion

Case study: Mars Pathfinder

- Within a few days of landing, when Pathfinder started gathering meteorological data, spacecraft began experiencing total system resets



Priority inversion

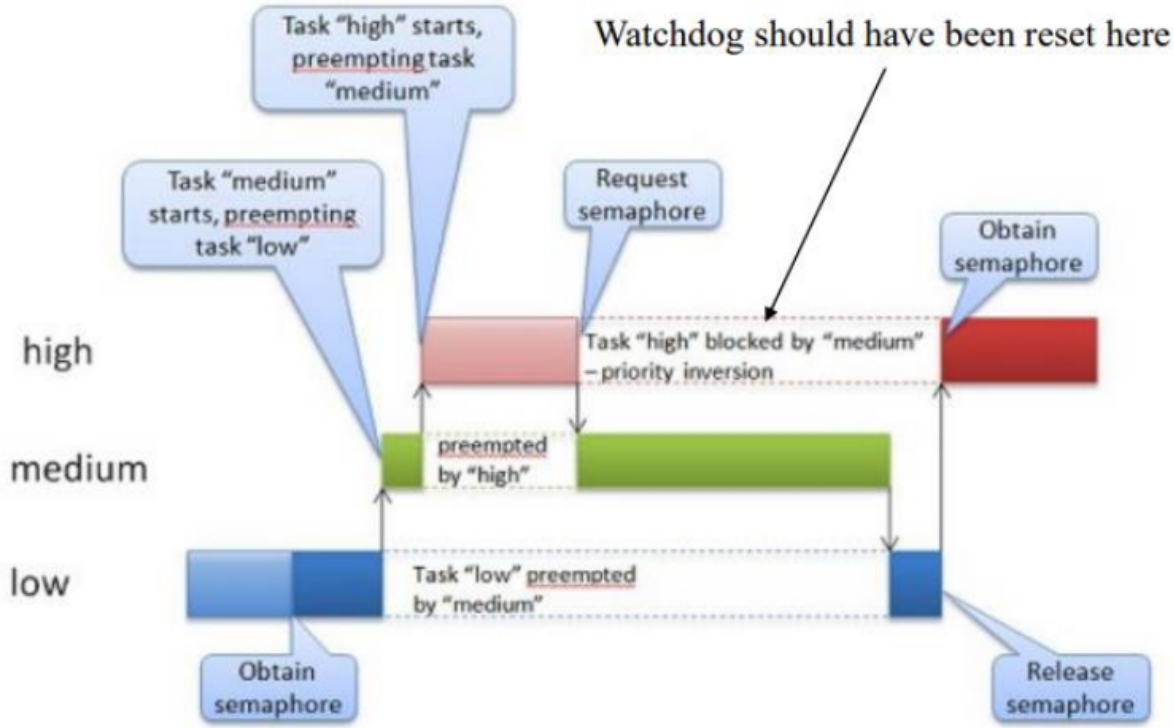
Case study: Mars Pathfinder

- NASA and JPL engineers had exact replica of the spacecraft in their lab
- After 18 hours of execution, early next morning when all but one engineer had gone home, the symptom was reproduced.
  - Watchdog timer wasn't being updated properly

Priority inversion

Case study: Mars Pathfinder

- Priority inversion bug prevented Watchdog update



Priority inversion

Solutions: dynamic priorities based on resource acquisition

- NPP
- BIP
- many others

Priority inversion

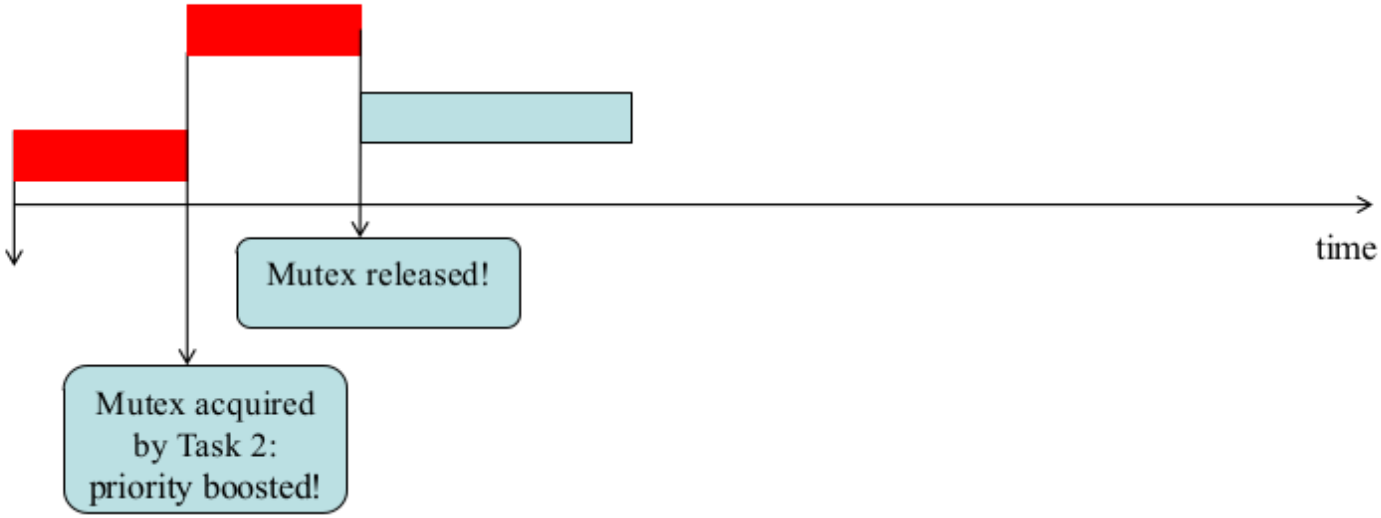
NPP: Non Preemption Protocol

- Whenever any task acquires a mutex, gains highest possible priority
- Goal: make it execute to release mutex as soon as possible

Priority inversion

NPP: Non Preemption Protocol

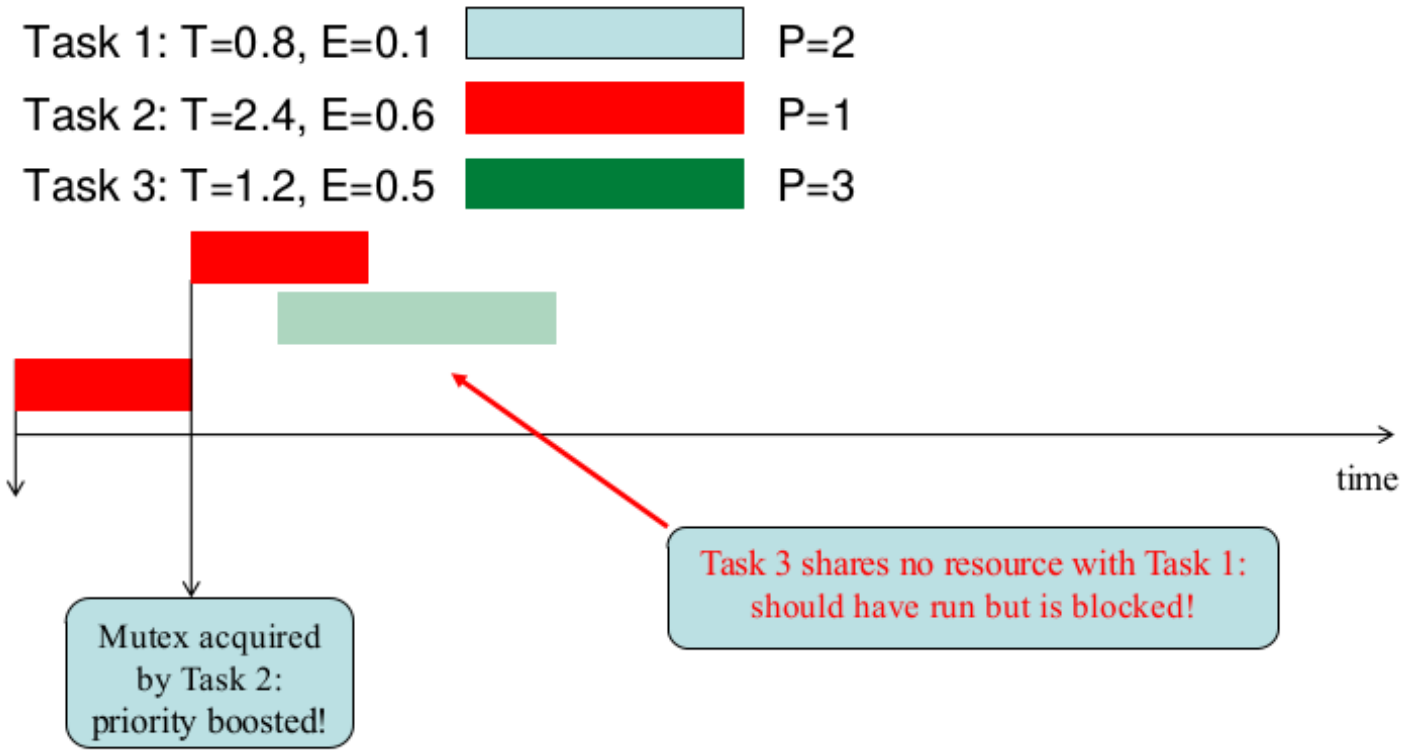
- Example (Working well: Task 1 and Task 2 share resource)
  - Task 1: T=0.8, E=0.1 P=3
  - Task 2: T=2.4, E=0.6 P=1
  - Task 3: T=1.2, E=0.5 P=2



Priority inversion

NPP: Non Preemption Protocol

- Example (blocking a task: Task 1 and Task 2 share resource)



Priority inversion

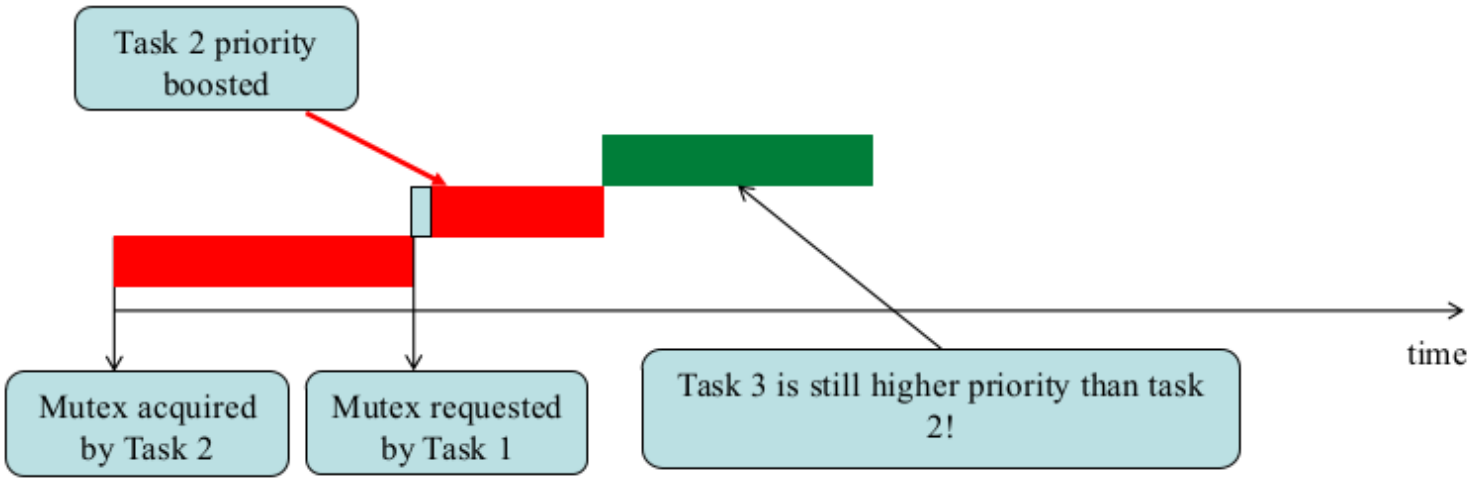
BIP: Basic Inheritance Protocol

- Task A acquires mutex
- Task B (where $P_B > P_A$) fails to acquire
- $P_A = P_B$

Priority inversion

BIP: Basic Inheritance Protocol

- Example (Working well)
  - Task 1: T=0.8, E=0.1 P=2
  - Task 2: T=2.4, E=0.6 P=1
  - Task 3: T=1.2, E=0.5 P=3



Priority inversion

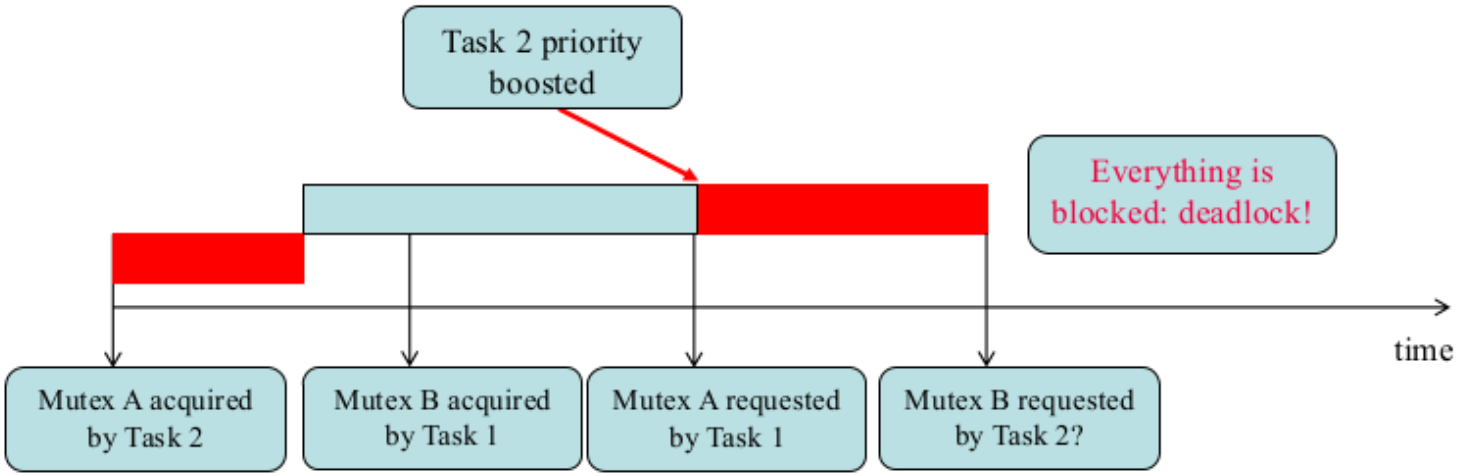
BIP: the problem

- 2 tasks require the same 2 mutexes
- But they request them in opposite order

Priority inversion

BIP: Basic Inheritance Protocol

- Example (Deadlock: 2 mutexes)
  - Task 1: T=0.8, E=0.1  P=2 (Mutexes A and B)
  - Task 2: T=2.4, E=0.6  P=1 (Mutexes A and B)
  - Task 3: T=1.2, E=0.5  P=3



Priority inversion

BIP: the problem

- Deadlock!

Priority inversion

HLP: Highest Locker's Priority

- Give a priority to each mutex
  - Defined as the highest priority of tasks that request it
- Whenever a task acquires mutex, it gains its priority

Priority inversion

HLP: Highest Locker's Priority

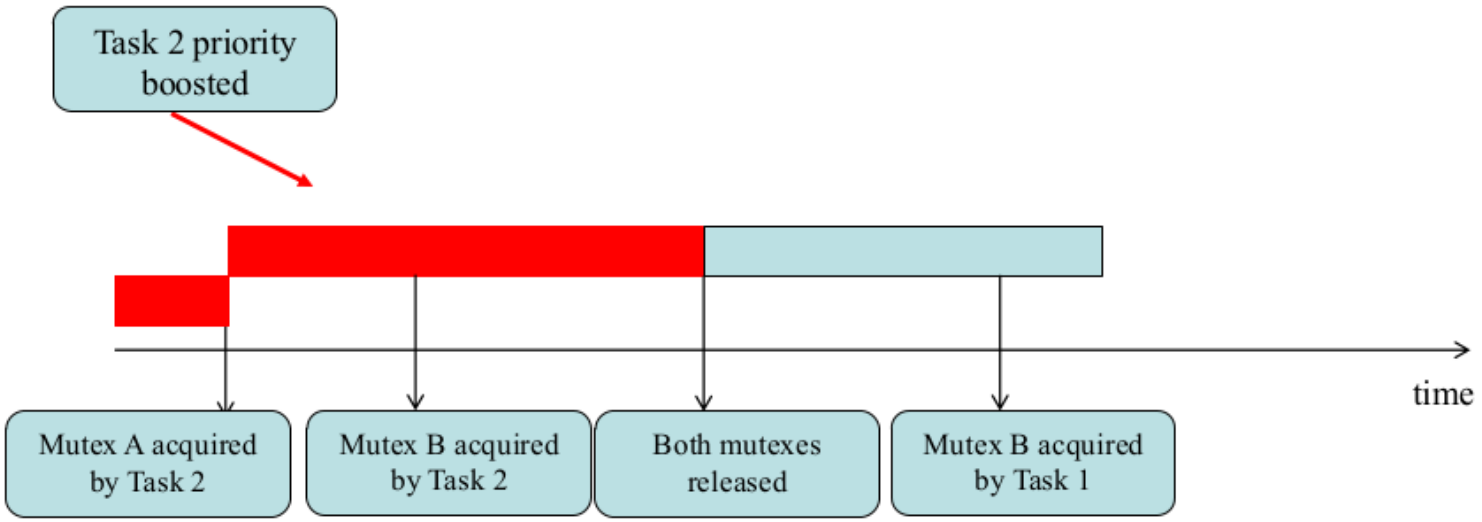
Task	Task Priority	Mutex used
4	4	M3
3	3	M1, M0
2	2	M1, M2
1	1	M2, M0

- Mutex priority:
  - M0 – 3
  - M1 – 3
  - M2 – 2
  - M3 - 4

Priority inversion

HLP: Highest Locker's Priority

- Task 1: T=0.8, E=0.1  P=2 (Mutexes A and B)
- Task 2: T=2.4, E=0.6  P=1 (Mutexes A and B)
- Task 3: T=1.2, E=0.5  P=3



Review

Aperiodic scheduling

- Aperiodic tasks
- Earliest Deadline First

Review

Challenges

- Race conditions
- Resource contention
- Priority inversion

Review

Take-home lesson

- Designing concurrent systems is hard

Next

Petri Nets

- A formalism for concurrent system design